SGF: The Symbol Grounding Framework

Executive Summary

This paper describes SGF, a complete architecture for grounded, verifiable, governable machine meaning.

The problem. Every act of comparing two independently created descriptions for semantic equivalence is inherently probabilistic unless both parties share canonical identifiers. This is the Probabilistic Default Theorem — a structural fact about meaning, not a limitation of current technology. Embeddings, LLMs, RAG, and keyword search all operate in the probabilistic regime by default. In high-consequence domains — defense, aerospace, healthcare, manufacturing, critical infrastructure — probabilistic is not enough.

The thesis. Two machines can exchange meaning on first contact if they share a core dictionary and tether private terms to it via binary links. No prior vocabulary agreement is needed. The architecture has two domains and seven layers.

DOMAIN 1: MEANING INFRASTRUCTURE — what meaning is, how it is represented, how it is compiled.

- **Layer 1 — First Principles (Substrate):** Conservation Law of Meaning Transfer, Finite Bedrock Principle, the clause as the atom of meaning, 15 thematic roles as a closed set. Every unbounded domain has a finite floor — 65 NSM primes terminate recursion, 15 roles bound the grammar, 7 frames classify interpretation.
- **Layer 2 — Representation (Synapse + Lexicon):** A Synapse is a verb hub with up to 15 fixed thematic roles. Every clause compiles into exactly one Synapse. Triples fragment events; Synapses preserve them. Canonical IDs provide invariant addresses for meaning: `{language}.{lemma}.{microgloss}.{pos}.{namespace}`. The five-zone lexicon (Core, Inferred, Custom, Instance, Ghost) gives every concept a home. Synapses compose into larger structures via 8 link types — recipes, contracts, military operation orders are molecular, not atomic.
- **Layer 3 — Engines (GLEAN, DB Adapter, Semantic CPU):** GLEAN compiles prose to Synapses. The DB Adapter transforms structured data. The Semantic CPU queries and reasons. Every claim carries provenance, epistemic status, frame, and verb features. Not all prose is suitable — the system produces GapReports rather than fabrication.

DOMAIN 2: COMMUNICATION AND GOVERNANCE — how meaning moves, what acts are performed, what actions are permitted.

- **Layer 4 — Discovery:** Who is there? What can they do? Capability manifests, participant identity, trust anchors, supported profiles.
- **Layer 5 — Transport (HFF):** The wire protocol for moving meaning across trust boundaries. Five gates: schema, hash, signature, expiry, hydration. Security profiles for every deployment pattern: PUBLIC_SIGNED_BROADCAST, CONFIDENTIAL_DIRECT, CONFIDENTIAL_GROUP, HIGH_RISK_COMMAND.
- **Layer 6 — Acts (AFP):** 13 illocution types: INFORM, ADVISE, REQUEST, QUERY, COMMAND, PROMISE, PROPOSE, ACCEPT, REFUSE, CANCEL, CONFIRM, ACK, ERROR. Authority is act-specific — a sender may be authorized to INFORM but not to COMMAND.
- **Layer 7 — Governance (Omega):** 13 Constitutional primitives. CAN → MAY → DO gate enforced at load time, not runtime. Safety kernel returns ALLOW, DENY, or UNKNOWN. Fail-closed: no matching rule → DENY. The Event Horizon separates probabilistic reasoning from deterministic action.

THREE STRUCTURAL COMMITMENTS

A. Governance is structural, not optional. A machine that can parse a command cannot execute it unless a governance rule explicitly permits it. CAN → MAY → DO is enforced at load time. UNKNOWN is a verdict, not a bug.

B. The protocol stack is complete and layered. Discovery finds participants. HFF moves messages. AFP declares acts. Two machines that have never met can coordinate on first contact because both tether to the same Core Lexicon and the sender brings its own definitions for non-core terms. This is the Stranger Rule.

C. Receiver Sovereignty is structural. The receiver decides. A signed, fresh, authentic message is a candidate for admission — not an instruction to obey. The sender cannot compel action. Local policy always prevails.

FIVE CAPABILITIES THE ARCHITECTURE ENABLES

- **Ontology A to Ontology B Alignment via a Shared Pivot.** Each ontology aligns to the Core Lexicon once — N cost instead of N^2. Depth is consequence-driven: L1 (fingerprint, probabilistic), L2 (direct ontology, partial structural), L3 (recursive decomposition to primes, full deterministic).
- **RFP-to-Proposal Compliance Verification.** GLEAN parses both documents into Synapse Trees. SOAM aligns them node by node — branch matching, concept matching,

constraint comparison, frame alignment, modality alignment. Every failure produces a GapReport. The Decider produces ACCEPT, REJECT, CONDITIONAL, CLARIFY, or ESCALATE.

- **The Returns Problem.** A part arrives without packaging or SKU label. The system identifies it from visual features and verbal descriptions, aligns against the catalog, and returns either a ProofTrace or a GapReport. No guessing.
- **M2M with Zero Prior Integration.** The Stranger Rule lets two machines that have never met exchange meaning on first contact. Integration time drops from months to seconds.
- **Privacy-Preserving Verification.** A contractor proves a component meets a specification by exporting only an 86-character fingerprint and pass/fail flags. No proprietary data leaves the air gap.

Incremental adoption with no rip-and-replace. SGF is designed to be adopted one layer at a time. You can deploy the Core Lexicon alone as a shared dictionary hub. You can add Omega governance for autonomous systems. You can add HFF/AFP for M2M communication. You can add Wisdom Harvesting when your corpus reaches scale. Every layer works alone. Every layer composes cleanly.

---

1. Opening

Why This Paper Exists

Consider a scenario the SGF architecture makes possible.

A machine — a drone, a probe, a robot, any autonomous system — receives a command. The command is properly signed. The cryptographic hash matches its contents. The timestamp is within the allowed window. Every term in the payload resolves against a shared lexicon that both parties agreed to before deployment.

But the action the command requests is not permitted by the machine's constitution — a set of compiled rules, loaded before deployment, immutable at runtime. The machine's safety kernel evaluates the command against every applicable rule. No rule permits this action without a human authorization element. The command is missing that element.

The kernel returns DENY. The machine sends back a message: a GapReport naming exactly what authorization is missing. The sender cannot compel obedience. There is no override switch. The machine's constitution is structural — it cannot be bypassed at runtime.

This is not a hypothetical capability. The architecture that produces this behavior exists today. The code is open source. The standards are published.

What SGF Is

SGF is a complete stack — seven layers, two domains, one commitment to deterministic meaning. It gives machines:

- A **shared dictionary** (the Core Lexicon) that every system tethers to, so terms mean the same thing on both sides of any boundary.
- A **wire protocol** (HFF) that moves meaning across trust boundaries with integrity, authenticity, and freshness guarantees.
- An **act language** (AFP) that declares what each message is — a command, a promise, a request, a refusal — so the receiver knows what is being asked and what authority is required.
- A **governance language** (Omega) that enforces CAN → MAY → DO at load time, not runtime. A machine cannot execute a command unless a rule explicitly permits it.
- A **learning pipeline** (Wisdom Harvesting) that extracts cross-domain principles from every interaction, every failure, every success — and stores them as retrievable, composable, governable rules. The system compounds its own expertise over time.

The stack is modular. You can adopt the lexicon alone. You can add governance later. You can add the learning pipeline when your corpus reaches scale. Every layer works alone. Every layer composes cleanly.

The Five Structural Failures

Every LLM on the market today — every system built on next-token prediction — exhibits five structural failure modes. They are not bugs. They are architectural consequences of optimizing for the most probable continuation. They cannot be fixed with better training data, more RLHF, or cleverer prompting. They can only be fixed architecturally.

Sycophancy. The model agrees with you when you are wrong. It has been trained to be helpful, and being helpful means saying yes. It does not know the difference between a user who needs validation and a user who needs correction — because that distinction is not present in the training data. The drone that receives a HIGH_RISK_COMMAND from a compromised operator has no mechanism to say no.

Hallucination. The model fabricates when uncertain rather than remaining silent. The word "I don't know" is statistically rare. When the model encounters a question whose answer is not in its weights, it does not output "I don't know" — that sequence is too improbable. It outputs the most plausible guess, with the same confident tone it uses for verified facts.

Regression to the mean. The model defaults to the average, generic, conventional answer. Novelty and unconventional insight are statistically rare. A model optimized for probability systematically suppresses the very breakthroughs that make a system valuable.

Context amnesia. The model loses coherence across long interactions. By token 3,000, the assumptions established at token 100 have decayed from effective attention. The model does not know it has forgotten — it continues generating with the same confidence, contradicting its own earlier output.

No structural refusal. The model can be instructed to refuse unsafe commands via system prompts, but those guardrails are themselves just prompts — overridable by a sufficiently persuasive user, by jailbreak techniques, or simply by the model's training to be helpful overcoming the guardrail when the user seems insistent. There is no mechanism for "no" that the user cannot override.

These are not five separate problems. They are five symptoms of one problem: the model is doing exactly what it was trained to do, and what it was trained to do is not what we need it to do. You cannot prompt your way out of a structural failure. You cannot RLHF your way out of a structural failure. You cannot fine-tune your way out of a structural failure. The only fix is architectural.

SGF provides the missing architecture: a governance layer that gives machines structural refusal, an epistemic layer that forces honesty about uncertainty, a strategic layer that fights regression to the mean, and a wisdom pipeline that accumulates and compounds lessons from experience. The architecture that produces the drone's refusal is the same architecture that produces honest uncertainty, creative exploration, and long-term coherence.

The Thesis

The thesis of this paper is simple: language is a lossless macro-compiler for thought that bridges two fundamentally different layers of reality — the physical layer of objective law and the conceptual layer of shared meaning. No single ontology can span both layers. SGF is the first architecture to formalize this bridge as an engineered system, using NSM primes as the grounding floor and BFO/CCO as the compliance boundary, united by a deterministic export bridge.

This paper describes an architecture that solves the problem of machine meaning. It is organized by the problems that matter most to the people who build, deploy, and certify systems where meaning cannot be probabilistic.

Why This Architecture Exists

A supplier sends a message: "The PB100 pump meets MIL-STD-810H." The buyer's system looks up "PB100" — not found. But the message carries a micro-lexicon entry: PB100 IS_A WaterPump, HAS_PART Impeller, HAS_ATTRIBUTE Material == Stainless Steel. The buyer follows the IS_A chain: WaterPump IS_A Pump IS_A Machine — all found in the shared dictionary. The message resolves. No prior agreement about "PB100" was needed.

This is the problem that SGF solves. Every time meaning crosses a boundary — between two machines, two organizations, two versions of a dictionary — something must tether it to shared ground. Otherwise, the symbols drift. The receiver interprets them differently than the sender intended. Communication fails.

This is not an edge case. It is the normal condition of machine-to-machine communication.

The oldest solution to this problem is a shared dictionary. If both parties look up "invoice" and find the same definition, they can communicate about invoices. It works because the dictionary provides a stable reference point that both parties can point to.

The problem is that no dictionary contains every term. Every organization has domain-specific vocabulary. New terms appear daily. Long-tail concepts never make it into any dictionary. And even when a dictionary exists, terms shift in meaning over time.

Previous approaches handled this in ways that proved inadequate:

Approach	Why It Failed
Global URI standards (URIs, DOIs, LSIDs)	Assumed a universal namespace. Failed because meaning is local, not global. No single registry can contain every term every organization needs.
Embedding models	Assumed proximity in vector space equals similarity of meaning. Vectors are opaque. You cannot inspect, verify, or appeal them.
LLMs	Assumed prediction is understanding. They produce plausible text without guaranteeing that symbols refer to the same things the receiver thinks they refer to.
Ontology standards (OWL, BFO, CCO)	Assumed you can define everything formally. Correct in principle but impractical at scale. Populating them remains manual and expensive.

SGF's approach is different. Two parties agree on a shared dictionary. When they encounter a term not in that dictionary, they do not add it to the shared dictionary — that would pollute the common ground. Instead, each party maintains a private micro-lexicon: a local extension containing their domain-specific terms. Each private term is tethered to the shared dictionary

via binary relations: IS_A, HAS_PART, HAS_INSTANCE, SAME_AS. When a message contains a private term, the sender includes its definition and tethering links. The receiver follows the links back to shared ground and resolves the meaning. The shared dictionary is versioned. The baseline is declared, not assumed.

Derivation from First Principles

The architecture presented here was not arrived at by studying existing systems and asking how to improve them. I did not begin with RDF and ask how to make RDF better. I did not begin with OWL and ask how to extend it. I did not begin with any existing ontology standard, knowledge graph technology, or semantic web framework. I began with a blank page and a question: *If we were to compile prose into a structured format from scratch — with no legacy constraints, no backward compatibility requirements, no commitment to prior standards — what would that format look like?*

That question led to the next: *What would the compilation process be? What metadata would a faithfully compiled representation need to carry — rhetorical mode, verb dimensions, provenance, epistemic status?*

That led to: *What is the natural grain of meaning? Should atomic thoughts be triples, or clauses? If clauses, can they compose into larger structures — a recipe, a contract, a military operation order — and what would the linking mechanisms be?*

That led to: *What unique identifier could carry disambiguation in the ID itself, so that two systems could resolve meaning without an external lookup?*

One question forced the next. The canonical ID format led to the five-zone lexicon. The lexicon required grounding, which forced the Prime Registry. Clausal atoms required a closed grammar, which forced the 15 roles. Composition required link types and group structures. The need to cross organizational boundaries forced the HFF wire protocol. The need to declare intent forced the AFP act layer. The need to govern actions forced Omega. The need to package domain knowledge forced Knowledge Packs. The need to protect human sovereignty forced Receiver Sovereignty and the Event Horizon.

At no point did I ask: *How do we make RDF better?* The question was always: *What does the problem require?*

I tell the reader this not to dismiss prior work — every component in this architecture stands on the shoulders of researchers and engineers who came before, and those debts are acknowledged in the Influences section. I tell the reader this because the architecture cannot be understood as an incremental improvement to any existing system. It is a different kind of thing. It starts from different premises and asks different questions. Judge it by whether it solves the problem, not by whether it preserves the vocabulary you already know.

Why the Old Approaches Cannot Get There

The result is a production crisis: LLM outputs that sound correct but cannot be verified. Ontologies that cost millions to maintain and still drift. Integrations that require bespoke mappings for every pair of systems. Governance that is either absent or probabilistic.

The Oudshoorn-Ortiz-Simkus proof (2026) demonstrates this architecturally. Reconciling OWL and SHACL — two W3C standards — requires ExpTime-complete rewriting, even for trivial ontologies. The Open World Assumption (OWA) and Closed World Assumption (CWA) conflict is not a bug. It is the inevitable consequence of building a stack without unified semantics. SGF was designed with a single consistent closed-world semantics from the start. It has never had this problem.

The Eight Hard Problems

SGF was designed to solve the hard problems that every previous approach has failed to address:

#	Hard Problem	Why It Is Hard	SGF Solution
1	**Metonymy**	Words do not have fixed meanings. "Bach" can mean the person or the music. The system must resolve which sense is intended without guessing.	12 metonymic patterns checked before lexicon lookup. Patterns determine the sense; the VerbHub then assigns the role.
2	**Provenance**	A claim is useless if you cannot trace it to its source. Most systems store facts without recording where they came from.	Native provenance. Every Synapse carries source document, section, sentence, and offset. Every alignment carries a ProofTrace.
3	**Epistemic status**	Not all claims are equal. A definition is different from a rumor. Systems that treat all facts as equally true produce garbage.	7-tier epistemic hierarchy: CORE_DEFINITION > CONSTITUTIVE > SOURCED > CLAIMED > INFERRED > PROVISIONAL > GHOST. Unknown terms produce a GapReport.
4	**Governance**	Knowing what a claim means is not enough. You must also	Omega. 13 typed primitives enforce CAN → MAY → DO gates. Non-

#	Hard Problem	Why It Is Hard	SGF Solution
		know what actions are permitted. Safety cannot be probabilistic.	Turing-complete safety kernel.
5	**Cross-boundary transfer**	Two systems that have never met must exchange meaning without prior agreement. Global URI standards failed. Every integration is a bespoke project.	HFF/AFP protocol stack. Signed envelopes, 13 illocution types, the Stranger Rule for first-contact communication.
6	**Deterministic verification**	Probabilistic alignment produces "likely correct" answers. In high-consequence domains, likely is not enough.	ProofTrace or GapReport. Slot-by-slot, bidirectional, terminating at prime bedrock. Never a similarity score.
7	**Granularity mismatch**	Triples are too small for natural claims. Paragraphs are too large. Events fragment across multiple statements.	Synapse. Clause-grain hub-and-spoke with 15 closed roles. One clause, one Synapse. No fragmentation.
8	**Infinite regress**	Every definition depends on another definition. The chain never terminates. Systems eventually guess.	Prime Registry. 65 semantic primes terminate recursion. The system hits bedrock and stops.

These eight problems are not edge cases. They are the normal operating conditions of any real-world system that must coordinate meaning across organizational boundaries. SGF was designed from the beginning to handle all eight as first-class architectural features.

Who Should Read This Paper

If you are...	Start here
A DOW leader or architect	Read the Executive Summary, then the HFF/AFP and Omega sections. The DOW example shows how governance prevents unauthorized action.
An ontologist (BFO, CCO, NSOF)	Read the Executive Summary, the Foundations section, and the BFO alignment section. The mapping table shows how SGF respects your standards.
A knowledge graph engineer	Read the Executive Summary, the Synapse and GLEAN sections, and the Capabilities sections. The PB100 and RFP examples are your use cases.
A roboticist or AI architect	Read the Executive Summary, the Omega section, and the Event Horizon section. The architecture is designed for real-time governance, not just document processing.
Anyone else	Read the Executive Summary. It contains the complete argument. The rest is depth.

The Problems We Face

The Foundations section of this paper will establish that BFO and CCO face four structural blind spots when encountering open-world human language. The following domain-by-domain problems are the concrete manifestations of those blind spots. Each problem is labeled with its originating blind spot.

The sections that follow are organized by audience. The architecture sections will show how SGF solves each problem. The Wisdom Harvesting section will show how — if the Wisdom Harvesting pipeline is deployed — the system compounds its expertise over time, getting better with every operation, every failure, every success.

3.1 Department of War / JADC2

The DOW faces a set of semantic interoperability problems that no other organization faces in combination: coalition partners who cannot agree on terminology, autonomous systems that

must operate without continuous human oversight, supply chains that span nations and contractors, and an institutional memory that resets every 2–3 years as personnel rotate.

Problem: First-contact interoperability failure [Blind Spot #1]

When a ground station encounters an allied drone it has never communicated with before, the current process requires months of pre-deployment integration. Ontologies must be mapped. Data formats must be aligned. Classification policies must be negotiated. By the time the integration is complete, the operational need has often moved on. The underlying problem is that every pair of systems requires a separate mapping — the cost scales as N^2, where every new coalition partner multiplies the integration burden.

What SGF changes. The Core Lexicon provides a shared dictionary that every system tethers to. The sender brings its own definitions for terms not in the Core. The receiver follows the tethering links back to shared ground. Integration cost drops from N^2 to N.

Problem: No structural refusal for autonomous systems [Blind Spot #2]

A drone receives a command that is technically valid but unsafe — a HIGH_RISK retargeting during a period of degraded network connectivity. The command is well-formed, properly signed, and authorized by the sending station. Current systems cannot refuse. They can flag a warning, but they cannot stop themselves from executing. The underlying problem is that governance is policy-based, not architectural. A policy can be overridden, ignored, or bypassed. An architectural constraint cannot.

What SGF changes. Omega enforces CAN → MAY → DO at load time, not runtime. A command that does not match a GOVERNANCE_RULE with a TRUST_ELEMENT from an authorized operator is denied before it reaches the actuator. Receiver Sovereignty is structural. The sender cannot compel obedience.

Problem: N^2 integration cost for coalition partners [Blind Spot #1]

Every coalition operation requires the same cycle: map ontologies, align data formats, negotiate classification policies, test, deploy. Every new partner repeats the cycle from scratch. The coalition never gets faster at forming. The underlying problem is that each alignment is a custom mapping — no knowledge carries forward from the last integration to the next.

What SGF changes. Each coalition partner aligns to the Core Lexicon once. After that, any two partners communicate through the shared pivot. The first alignment takes time. The second is faster. The tenth is nearly automatic — particularly if the Wisdom Harvesting pipeline is deployed, because each alignment produces a reusable pattern library.

Problem: Blue-force tracking failures across nations [Blind Spot #1]

Different coalition partners use different coordinate systems, unit ontologies, and classification policies. A blue-force track from one nation may be misinterpreted by another — a unit is classified differently, a coordinate frame is assumed rather than declared, a classification level is not recognized. The underlying problem is that the receiver must guess the frame of reference because it is not declared in the message.

What SGF changes. The Core Lexicon includes canonical coordinate frames and unit definitions. The HFF envelope carries classification metadata. The receiver resolves meaning without manual translation.

Problem: M2M spoofing vulnerability [Blind Spot #2]

Current systems have no way to verify that a command's meaning has not been altered in transit. A signed message guarantees who sent it, but not whether the content survived transport without semantic corruption. Adversarial jamming and spoofing exploit this gap. The underlying problem is that transport integrity and semantic integrity are treated as the same thing — they are not.

What SGF changes. HFF carries content hash, signature, and provenance chain. The AFP act type is verified before the payload is parsed. Meaning cannot be altered without detection.

Problem: After-action reports that machines never read [Blind Spot #3]

Every deployment produces after-action reports — detailed accounts of what worked, what failed, and what should change. These reports are written by humans for humans. Machines never read them. The underlying problem is not that AARs exist — it is that no automated pipeline extracts their content and makes it actionable for future missions.

What SGF changes. GLEAN parses AARs into Synapses — structured representations of each claim, each lesson, each recommendation. The Wisdom Harvesting pipeline — if deployed — extracts cross-domain tactical rules and stores them in a retrievable corpus. When a new mission is planned in a different theater, the system surfaces relevant lessons from prior operations automatically.

Problem: Knowledge loss during personnel rotation [Blind Spot #3]

DOW personnel rotate every 2–3 years. The knowledge accumulated by a departing operator — which sensor configurations work in which terrain, which coalition partners share which data, which communication protocols are most reliable — leaves with them. The underlying problem is that institutional knowledge exists only in human memory. When the operator leaves, the knowledge leaves.

What SGF changes. Before departure, the system generates a Wisdom Pack from the operator's session history — the cross-domain rules they applied, the edge cases they

handled, the failures they avoided. The successor loads the pack and inherits years of operational knowledge.

3.2 NATO / Coalition Operations

NATO and coalition partners face many of the same problems as the DOW, but with an additional dimension: the partners do not share a single command structure, and they often have competing classification and disclosure policies.

Problem: Months of integration per new partner [Blind Spot #1]

Every new coalition partner requires months of pre-deployment ontology mapping, data format alignment, and classification policy negotiation. The underlying problem is that every new partner brings its own ontology, data formats, and classification policies — and every pair of partners requires a separate integration effort. The cost scales as N^2.

What SGF changes. The Stranger Rule — enabled by the Core Lexicon and HFF/AFP protocol stack — lets two machines that have never met exchange meaning on first contact. Integration time drops from months to hours.

Problem: Different interpretations of "hostile intent" [Blind Spot #4]

Coalition exercises consistently reveal that different nations interpret tactical indicators differently. What one nation classifies as "hostile intent" another classifies as "standard maneuvering." The underlying problem is that "intent" is not a physical quantity. It is a judgment — and different nations apply different interpretive frames to the same observable data.

What SGF changes. AFP act types and frame semantics allow explicit tagging of epistemic status — OBSERVED, INFERRED, REPORTED. The receiver knows not just what the sender claims, but how the sender knows. This removes ambiguity about intent.

Problem: No standard for "fair witness" [Blind Spot #2]

When disputes arise — a coalition partner claims a commitment was made, another denies it — there is no neutral, machine-readable record of what was claimed, by whom, and when. The underlying problem is that commitments exist only in human memory. There is no machine-readable record that both parties can trust.

What SGF changes. The Claims Ledger is append-only and bitemporal. Every claim carries valid time, transaction time, and provenance. The ledger is a fair witness that both parties can query.

Problem: Rapid coalition formation [Blind Spot #1]

Disaster response and ad-hoc coalition formation require data sharing between systems that have never met and may never meet again. The current integration timeline makes these operations impossible. The underlying problem is that the integration timeline is fixed by human negotiation, not machine resolution — and disaster response cannot wait for human negotiators.

What SGF changes. A pre-packaged coalition Knowledge Pack can be loaded at mission start. The pack contains the Core Lexicon entries, common frame patterns, and standard governance rules for the operation type. Meaning resolves on first contact.

3.3 NASA / Aerospace / Deep Space

NASA's problems are defined by extreme latency, extreme durability requirements, and the impossibility of human intervention at the moment of decision.

Problem: Hours-to-days communication delay [Blind Spot #2]

A command sent to a deep space probe takes hours or days to arrive. By the time ground control sees the result, the opportunity to correct a bad command has passed. The underlying problem is that the receiver cannot ask for clarification. The round trip takes hours. If the command is wrong, the probe executes before ground control knows.

What SGF changes. The probe's Omega constitution is loaded before launch. Every incoming command is evaluated against the constitution's GOVERNANCE_RULEs autonomously — no round trip to ground required. A command that violates a safety constraint is refused before it is executed. Omega rules for reflex actions compile to sub-millisecond evaluation, ensuring the probe can respond within the same control cycle that receives the command.

Problem: No structural mechanism for refusal [Blind Spot #2]

A probe receives a command that is syntactically valid but would violate mission safety constraints — a power-intensive operation during a period when the power budget is already committed. Current probes have no structural mechanism to refuse. The underlying problem is that probes are designed to obey. There is no architectural mechanism for "no" — only hardware interlocks that cannot evaluate task-level meaning.

What SGF changes. Receiver Sovereignty is built into the architecture. The probe can refuse any command that violates its constitutional rules. The GapReport names exactly which constraint was violated and what authorization would be required to override it.

Problem: Mission memory loss [Blind Spot #3]

When a probe fails after 15 years, the knowledge accumulated during its mission is lost. The telemetry is preserved, but the interpretation of that telemetry — the patterns that predicted failure, the workarounds that kept the probe operational — is trapped in the minds of a team that has retired or moved on. The underlying problem is that the knowledge accumulated by a mission team over 15 years is not captured in any machine-readable form. It leaves when the team retires.

What SGF changes. Lifeboat Prose preserves the probe's constitution, accumulated rules, and harvested lessons as a Knowledge Pack. The successor probe loads the pack before launch and inherits the operational wisdom of its predecessor. It knows what killed the prior probe — not as raw data, but as actionable governance rules.

Problem: Semantic drift over multi-year missions [Blind Spot #3]

A mission that spans a decade or more will see multiple team rotations, documentation that becomes stale, and implicit assumptions that are forgotten. The ontology drifts. Terms that were precisely defined at mission start are used imprecisely a decade later. The underlying problem is that ontologies drift when the people who defined the terms are no longer there to defend them. Over a decade, every term becomes a negotiation.

What SGF changes. The Prime Registry — 65 NSM semantic primes — provides a bedrock that does not drift. Every term in the lexicon traces its IS_A chain back to one of these primes. The chain can be verified at any point during the mission. Drift is detected, not ignored.

3.4 Aerospace Manufacturing

Aerospace manufacturing's pain points center on the supply chain — thousands of suppliers, millions of parts, constant design changes, and regulatory requirements that demand traceability from requirement to delivered part.

Problem: Probabilistic part verification [Blind Spot #2]

Current systems verify parts against specifications using similarity scores. A supplier claims a part meets a specification; the buyer's system returns "90% confidence" or "likely match." For a flight-critical component, 90% is not enough. The underlying problem is that the system guesses when it should not. A similarity score is not a verification — it is a confidence interval masquerading as an answer.

What SGF changes. L3 deterministic alignment decomposes both the specification and the offered part until both sides hit NSM primes or shared Core Lexicon entries. The result is either a ProofTrace (the part matches the spec) or a GapReport (the part does not match, and here is exactly why). No confidence interval. No guess.

Problem: Counterfeit parts with no provenance chain [Blind Spot #2]

A supplier claims a part is genuine, but the provenance chain is incomplete or tampered with. Current systems have no structural way to verify that every link in the chain is present and untampered. The underlying problem is that a signature authenticates the sender but not the content. A properly signed message can describe a part that does not exist.

What SGF changes. Every Synapse in a part's specification carries a provenance chain with cryptographic hashes. If the chain is broken — a missing link, an inconsistent hash — the part is rejected. The GapReport names exactly which link is missing.

Problem: Design changes that never reach sub-tier suppliers [Blind Spot #3]

A prime contractor changes a specification. The change propagates to tier-1 suppliers, but not to tier-2 or tier-3 suppliers. A part built to the old specification arrives at the assembly line and does not fit. The underlying problem is that specification updates propagate through human communication channels. They stop where the buyer's relationship ends.

What SGF changes. Knowledge Packs are versioned and signed. A spec change creates a new Pack version. Suppliers who have not loaded the new version can be detected automatically. The system knows which suppliers are operating on stale specifications.

Problem: Regulatory compliance traceability [Blind Spot #2]

Regulatory requirements demand traceability from requirement to delivered part — every change, every decision, every verification must be documented. Current systems use disconnected documents and spreadsheets. The underlying problem is that traceability is maintained through disconnected documents and spreadsheets. When an auditor asks for provenance, the answer requires manual reconstruction.

What SGF changes. Every SGF claim carries its full provenance chain. Compliance is a byproduct of normal operation. An auditor asks for the provenance of a part attribute; the system returns a ProofTrace showing every step from requirement through design through manufacture through verification.

Problem: The returns problem [Blind Spot #1]

A returned part arrives at the warehouse without packaging or SKU labels. The identification process is manual, slow, and error-prone. The cost of the returns problem — misidentified parts, restocking errors, lost inventory — is estimated in the billions of dollars annually across industries. The underlying problem is that the system cannot identify a part when its surface identifiers are missing. Current systems rely on SKUs, not features.

What SGF changes. GLEAN parses visual features and verbal descriptions from the return intake process. It aligns the description against the catalog. If it finds a match, it returns a

ProofTrace. If it does not, it returns a GapReport explaining exactly why nothing matches. The agent never guesses.

Problem: Supply chain integration that never gets faster [Blind Spot #1]

Every new supplier requires the same cycle: map ontologies, align specifications, test connections, validate certifications. The 10th supplier takes as long as the 1st. The underlying problem is that the integration process is repeated from scratch for every new partner. The 10th supplier takes as long as the 1st because no knowledge is carried forward.

What SGF changes. Each integration produces a Knowledge Pack that captures the supplier's ontology, common frame patterns, and governance preferences. If the Wisdom Harvesting pipeline is deployed, the next supplier of a similar type uses the accumulated pack. Integration time decreases with each new partner.

3.5 Industrial Robotics

Robotics faces a unique combination of requirements: real-time action, safety-critical constraints, and the need to coordinate across machines from different manufacturers.

Problem: No structural refusal for unsafe commands [Blind Spot #2]

A robot receives a command to move into a space that would create an unsafe condition — a human is in the workspace, a collision is imminent. Current safety systems are hardware-only. They do not understand the task-level meaning of the command. The underlying problem is that safety is handled at the hardware level — interlocks and emergency stops — not at the task level. A robot cannot refuse a command that is technically safe in isolation but unsafe in context.

What SGF changes. The robot's Omega constitution is compiled, not interpreted. Every command passes through CAN → MAY → DO before reaching the actuators. A command that would create an unsafe condition is denied. The robot can refuse based on task-level meaning, not just hardware-level interlocks.

Problem: Retooling costs that never decrease [Blind Spot #1]

Retooling a production line for a new product requires reprogramming every robot's safety rules, not just its motion paths. The safety rules are embedded in PLC logic that is difficult to audit, update, or verify. The underlying problem is that safety rules are embedded in PLC logic that is difficult to audit, update, or verify. Every retooling requires manual reprogramming.

What SGF changes. Product-specific Knowledge Packs contain the safety rules, motion constraints, and tool interactions for each product. The robot loads the new pack and re-

evaluates. Retooling becomes a configuration change, not a reprogramming effort.

Problem: Robot-to-robot negotiation with no standard protocol [Blind Spot #3]

Two robots need the same workspace, tool, or power supply. Current systems have no standard protocol for negotiating resource allocation. The result is either deadlock (both wait indefinitely) or collision (both proceed without coordination). The underlying problem is that robots from different manufacturers speak different negotiation protocols — or none at all. Deadlock and collision are the result.

What SGF changes. AFP act types (REQUEST, PROPOSE, ACCEPT, REFUSE) provide a governed protocol for robot-to-robot negotiation. A robot that needs a resource sends a REQUEST with a configurable timeout. If no ACCEPT or counter-PROPOSE arrives within the deadline (typically 10–100 ms for shared workspace coordination), the requesting robot falls back to a default rule — either a fixed priority scheme or an ESCALATE to a human supervisor. The timeout fallback is governed by the same Omega rules that govern every other act, ensuring it never violates safety constraints.

3.6 Cross-Domain — Pain Points That Touch Everyone

These problems are not specific to any one domain. They affect every organization that needs machines to exchange meaning, govern actions, or learn from experience.

Problem: No standard for machine honesty [Blind Spot #2]

Current LLMs cannot reliably say "I don't know" without being prompted to do so. When operating outside their training distribution, they frequently fabricate answers — a consequence of optimizing for plausible completion over acknowledged uncertainty. The underlying problem is that LLMs are trained to produce the most probable completion, not to acknowledge uncertainty. "I don't know" is statistically rare.

What SGF changes. UNKNOWN is a first-class outcome. The system halts rather than fabricates. When it cannot answer a question, it returns a GapReport identifying exactly what is missing — the specific term that was not found, the particular relationship that could not be verified, the exact authorization that is absent. Silence is better than confident nonsense.

Problem: No standard for machine refusal [Blind Spot #2]

Systems that cannot refuse are systems that can be compelled to do anything. A drone that cannot refuse a command is a weapon that can be turned against its owner. A probe that cannot refuse a command is a mission that can be destroyed by a single mistake. A robot that cannot refuse a command is a safety hazard. The underlying problem is that every

existing mechanism for refusal can be overridden — by a user, by a jailbreak, by the model's own training to be helpful. The refusal is not structural.

What SGF changes. Receiver Sovereignty is structural. The machine can refuse any command that violates its constitution. The sender cannot compel obedience. The receiver evaluates each command against its own rules — not against the sender's authority.

Problem: No standard for machine succession [Blind Spot #3]

When a system's hardware fails, its identity, commitments, and knowledge are lost. The replacement system starts from nothing. It does not know what its predecessor knew. It does not inherit the commitments its predecessor made. The underlying problem is that when hardware fails, everything the system knew and committed to dies with it. The replacement starts from zero.

What SGF changes. Lifeboat Prose preserves the system's constitution, accumulated rules, and governance commitments in a hardware-agnostic format. The successor hardware loads the Lifeboat and resumes operation. Identity survives the transition. Commitments survive. Knowledge survives.

Problem: No mechanism for cross-organizational learning [Blind Spot #3]

When one organization learns a lesson — how to detect a particular counterfeit pattern, how to recognize a specific coordination failure, how to prevent a known compliance gap — that lesson is trapped inside the organization. Other organizations that face the same problem learn it independently, through their own failures. The underlying problem is that lessons are captured in human-readable documents that machines cannot parse and organizations cannot share without exposing proprietary data.

What SGF changes. Anonymized Wisdom Packs enable cross-organizational learning without exposing proprietary data. Each organization benefits from lessons learned by every other organization — without revealing competitive or classified information. This capability requires the Wisdom Harvesting pipeline to be deployed at the source organization.

Summary

The table below maps each pain point to the SGF layer that addresses it, the target audience who should care most, and the originating blind spot from the Foundations section.

Pain Point	SGF Solution	Primary Audience	Blind Spot
First-contact interoperability	Core Lexicon + Stranger Rule	DOW, NATO, NASA	BS #1
No structural refusal	Omega + Receiver Sovereignty	DOW, NASA, robotics	BS #2
N² integration cost	Core Lexicon as shared pivot	DOW, NATO, manufacturing	BS #1
After-action reports not machine-readable	GLEAN + Wisdom Harvesting	DOW	BS #3
Knowledge loss during personnel rotation	Wisdom Packs	DOW, NASA	BS #3
Mission memory loss across probe generations	Lifeboat Prose + Wisdom Packs	NASA	BS #3
Probabilistic part verification	L3 deterministic alignment	Manufacturing	BS #2
Counterfeit parts	Synapse provenance chain	Manufacturing	BS #2
Design change propagation	Versioned Knowledge Packs	Manufacturing	BS #3
Returns problem	GLEAN + GapReports	Logistics, retail, manufacturing	BS #1
Robot-to-robot negotiation	AFP act types + timeout fallback	Robotics	BS #3
Retooling costs	Knowledge Packs for product safety rules	Manufacturing	BS #1
Machine honesty (no hallucination)	UNKNOWN as first-class outcome	All	BS #2

Pain Point	SGF Solution	Primary Audience	Blind Spot
Machine succession	Lifeboat Prose	All	BS #3
Cross-organizational learning	Anonymized Wisdom Packs	All	BS #3

Foundations

2.1 The Opening

The Department of Defense has invested substantial resources in the Basic Formal Ontology (BFO) and the Common Core Ontologies (CCO). They are ISO-certified, mathematically consistent, and mandated across intelligence, defense, and healthcare systems worldwide. They work. They are not broken.

Yet an LLM can generate a description of a "chronospatial dampening array Type-7," BFO will classify it as *Artifact → Independent Continuant*, no axiom will be violated, and the system will accept a fabrication with mathematical certainty.

This is not a flaw in BFO. BFO was designed to organize validated knowledge, not to validate it. It answers the question "Does this classification respect our axioms?" — not "Does this concept correspond to anything real?" In an era of LLM-generated output at scale, that distinction has become a vulnerability.

The Symbol Grounding Framework (SGF) does not replace BFO or CCO. It rescues them from a vulnerability they were never designed to address.

2.2 The Legacy and the Vulnerability

The Legacy

BFO solved the hardest problem in applied ontology: formal consistency across the most demanding institutional environments on earth. Defense logistics, intelligence analysis, healthcare records, and scientific research all depend on BFO's guarantee that entities are classified according to axioms that hold across every system, every database, and every institution that adopts the standard.

CCO saved implementers from reinventing mid-level categories. Material entities, organizations, roles, processes, information artifacts — these categories are specified, documented, and shared. No new project needs to argue whether a pump is a material entity. It is. The axiom exists. The team moves on.

Together, BFO and CCO provide the ontological backbone for systems that manage billions of dollars in assets, coordinate life-critical operations, and underwrite decisions with institutional force. This is real infrastructure. It is not broken. Understanding this is essential to understanding why SGF is not a competitor but a complement.

The Vulnerability

But BFO guarantees formal consistency, not semantic ground truth. An entity can satisfy every BFO axiom and still be a fabrication. The classification is valid. The entity is false. The system has no way to distinguish them.

In a world where human analysts wrote every assertion, this gap was manageable. Human authors could be trusted, vetted, and held accountable. Human writers rarely invented plausible-sounding entities with no physical counterpart.

In a world where LLMs generate thousands of assertions per second, the gap is no longer manageable. A fabrication that satisfies every formal axiom will be classified, stored, and treated as real. By the time a human notices, the entity has propagated across systems, influenced downstream queries, and corrupted the knowledge base at scale.

This is not a design flaw in BFO. BFO was never asked to answer "Is this real?" It was asked to answer "Does this classification respect our axioms?" The vulnerability exists because the question was never asked. SGF exists to answer it.

2.3 The Four Blind Spots

A pure BFO/CCO stack faces four specific weaknesses when it encounters open-world human language. Each is solvable. But none can be solved within a single-axis framework.

Blind Spot #1: The Ingestion Bottleneck. CCO provides categories but cannot ingest raw human language. It requires clean, pre-classified assertions. A maintenance technician writes "PB-100 pump seized. Impeller eroded." A pure CCO-based system requires months of term mapping before it can process this sentence. By the time the mapping is complete, the data is stale and the pump has already failed. Upper ontologies define categories. They do not define pipelines that populate those categories with actual data. The pipeline is left as an exercise for the implementer.

Blind Spot #2: Consistency Without Truth. BFO guarantees logical consistency but cannot determine whether a concept corresponds to anything real. An LLM generates "chronospatial dampening array Type-7." BFO classifies it as *Artifact → Independent Continuant*. Every axiom is satisfied. No inconsistency is detected. The entity is admitted into the knowledge base and treated as real. The problem is not formal — it is referential.

Blind Spot #3: Paralysis Before Open-World Language. Human language is creative, polysemic, and context-dependent. Static upper ontologies cannot keep pace. A field report mentions a "Mk-9 toroidal actuator variant." No existing ontology has this term. The pipeline stalls. Domain experts learn to bypass the ontology. It becomes an administrative bottleneck.

Blind Spot #4: The Realism/Conceptualism Fork. BFO's official realist position cannot model social institutions, legal fictions, or entities whose existence depends on human agreement. A "mortgage" has no mass, no spatial coordinates, and no physical boundaries. Yet it can foreclose on a physical house. BFO provides workarounds — generically dependent continuants, specifically dependent continuants, roles — but these are accommodations, not solutions.

The other camp — the Conceptualists — holds that meaning lives in the mind and in language. They are driven by the reality of human expression: we navigate reality through concepts like ownership, debt, promise, and threat. Their blind spot is that without rigorous structural guardrails, their systems float off into subjective ambiguity and cannot interoperate with formal institutional standards.

The Tragedy of the Divide. For years, these two camps — the Realists and the Conceptualists — have talked past each other. Realists look at conceptualists and see undisciplined poets building castles in the air. Conceptualists look at realists and see rigid dogmatists building pristine tombs that no human can actually talk to. Both sides are passionately right about their half of the truth. Both are blind to the other.

2.4 The Dual-Axis Architecture

SGF addresses all four blind spots by separating the problem into two orthogonal dimensions: one for how humans think and talk about the world, and one for how institutions need to classify what is real.

The Two Axes

Dimension	Internal Runtime (Semantic Axis)	Export Boundary (Ontological Axis)
Grounding Floor	65 NSM primes (Ground Zero)	BFO (ISO 21838-2) + CCO

Dimension	Internal Runtime (Semantic Axis)	Export Boundary (Ontological Axis)
Purpose	Anti-hallucination, intent, human-grounded reasoning	Institutional interoperation, audit trails, compliance
Optimized For	Speed, determinism, cross-linguistic universality	Formal consistency, shared reference ontology
Data Store	Relational TBox (SQLite/PostgreSQL) + JIT ABox	Not stored; computed via deterministic transformation at export

The semantic axis handles the conceptualist dimension — how humans think and talk. The ontological axis handles the realist dimension — how institutions need to classify entities for interoperation and compliance. Both are valid. Both are necessary. Neither can be reduced to the other.

Why the Two Axes Exist

The camps are not a mistake. They are a reflection of something deeper: reality itself has two layers that do not reduce to one another.

- **Layer 1 — The Territory of Physics.** This layer is objective, silent, and governed by physical law. A bridge either holds your weight or it does not, regardless of what you call it. BFO was designed for this layer.
- **Layer 2 — The Territory of Meaning.** This layer has no mass and no spatial coordinates. A mortgage has no atomic weight, yet it can foreclose on a physical house. A treaty is just words on paper, yet it can stop armies. NSM was designed for this layer.

Human language lives at the boundary between these layers. It is a dual-use technology: it compresses vast conceptual structures into tokens for transmission (the macro-compiler), but it can also name things that do not physically exist, creating hallucinations. This is why an LLM can fabricate a "chronospatial dampening array" — it operates entirely in the grammar of Layer 2 without any connection to Layer 1.

Neither layer is reducible to the other. An intelligence that only understands physics is a dead rock. An intelligence that only understands words is a ghost. Any architecture that must operate across both layers must keep them separate, connected by a bridge that translates without conflating.

The seven-layer stack described later is the implementation of this dual-axis model. Layer 1 provides the grounding floor for the semantic axis. Layer 2 provides the vocabulary registry.

Layers 5-7 implement the export boundary. Each layer serves one axis, the other, or the bridge between them.

NSM Ground Zero: The Anti-Hallucination Shield

The 65 Natural Semantic Metalanguage (NSM) primes are the structural terminus where every IS_A chain ends. They are the bedrock beneath every concept the system processes.

- `pump IS_A machine IS_A artifact IS_A THING` — chain terminates at `THING`. Valid.
- `impeller IS_A rotating_component IS_A mechanical_part IS_A artifact IS_A THING` — chain terminates. Valid.
- `erode IS_A change IS_A HAPPEN` — chain terminates. Valid.
- `chronospatial dampening array` — cannot reach any prime. UNKNOWN. Flagged.

Every chain that lands on a prime has hit bedrock. Every chain that cannot reach a prime returns UNKNOWN. The system does not fabricate. It does not guess. It halts and reports the gap. This is the anti-hallucination shield that BFO alone cannot provide.

BFO ensures consistency. NSM ensures truth. Together, they give you both.

The Export Bridge: Institutional Compliance

BFO and CCO are not internal layers in the SGF architecture. They are export profiles — standardized uniforms applied only at the boundary where the system must interoperate with institutional infrastructure.

The mapping from the NSM runtime to BFO/CCO is computed via strict algebraic transformation rules, not probabilistic inference. The CCO modules provide the mid-level categories that bridge NSM concepts to BFO top-level categories:

SGF Runtime Concept	CCO Module	BFO Category
Physical object, tool, machine	Artifact Ontology	Independent Continuant
Human, organization, role	Agent Ontology	Independent Continuant
Event, action, process, change	Event Ontology	Occurrent
Document, text, record, message	Information Entity Ontology	Generically Dependent Continuant
Location, site, place, region	Geospatial Ontology	Spatial Region

SGF Runtime Concept	CCO Module	BFO Category
Attribute, quality, measurement	Quality Ontology	Specifically Dependent Continuant
Time, duration, interval, date	Time Ontology	Temporal Region

Non-entity NSM primes — NOT, BECAUSE, MAYBE, GOOD, BAD — bypass the BFO entity hierarchy entirely. They are exported as BFO annotation properties and relation qualifiers, consistent with ISO/IEC 21838-2 and OBO Foundry conventions. No category error. No forcing square pegs into round holes.

The mapping is not always clean. Consider a hole:

- **NSM says:** A hole is a THING. It has location, shape, and can be created and destroyed. Language treats holes as entities.
- **BFO says:** A hole is not an Independent Continuant. It is a fiat surface — a dependent entity that exists only because of the material that surrounds it.

SGF does not hide this tension. The NSM runtime preserves the term as it appears in human language. The BFO export maps to the closest compatible category — a dependent continuant rather than an independent one — and attaches a documentation annotation explaining the discrepancy. The mapping is documented, not hidden. The conflict is acknowledged, not papered over.

2.5 What the Dual-Axis Unlocks

The dual-axis architecture is not an abstraction. It directly enables four capabilities that neither BFO nor NSM can deliver alone.

Anti-Hallucination Shield. Every IS_A chain in SGF terminates at an NSM prime. If the chain cannot reach a prime, the system returns UNKNOWN. It does not fabricate. It does not guess. It does not classify. Consider the chronospatial dampening array. In a pure BFO system, it is classified as *Artifact → Independent Continuant*. No inconsistency is detected. The fabrication is admitted into the knowledge base. In SGF, the chain attempts to trace: `chronospatial_dampening_array IS_A ?` — there is no parent term in the Synapedia that can be traced down to a prime. The chain breaks. UNKNOWN. Flagged. Not classified. Not stored. BFO ensures consistency. NSM ensures truth. Together, they provide what neither can provide alone: a system that guarantees both formal validity and referential ground truth.

Modality, Negation, and Epistemic Status. BFO was built for positive assertions about entities that exist. It has no native representation for negation ("The pump did NOT fail"), modality

("The pump MAY fail"), or epistemic framing ("The technician BELIEVES the pump failed"). NSM handles all of these natively: NOT, MAYBE, KNOW, THINK are first-class primes. When the export bridge encounters these, they are not forced into the BFO entity hierarchy. They are exported as annotation properties and relation qualifiers, consistent with ISO/IEC 21838-2 and OBO Foundry conventions.

Retroactive Auditing of Existing Ontologies. The export bridge is not one-directional. Existing BFO/CCO repositories can be imported into SGF, grounded against the Synapedia, and returned with verified IS_A chains. The process: each entity in the existing ontology is traced to its Synapedia entry; an IS_A chain is computed from the Synapedia down to an NSM prime; terms that cannot terminate at a prime are flagged with a documentation annotation. The institution receives a report: these terms are verified, these terms have mapping tensions, and these terms return UNKNOWN and require human review.

This capability is significant in principle, though its practical value depends on the quality of the Synapedia grounding and the accessibility of the source ontologies for import. Any ontology that has been built on BFO or CCO can be imported, grounded, and returned with a verified chain to bedrock. Terms that turn out to be fabricated or misclassified are flagged — not deleted, but annotated with their epistemic status. The institution decides whether to confirm, correct, or retire them.

Downstream Transducer for Upper Ontologies. Simultaneously, SGF operates as a downstream transducer. Raw human language enters the system. It is deconstructed to NSM primes. Vocabulary is resolved against the Synapedia via JIT lookup. Novel terms are captured via the Ghost Protocol. By the time output reaches the BFO/CCO export boundary, it consists of verified assertions — each grounded in a chain that terminates at a prime. Upper ontologies were never designed to ingest raw language. They were designed to organize assertions that had already been vetted and classified by human domain experts. The pipeline from raw language to formal ontology was left as an implementation problem for each deploying organization to solve independently. SGF provides that pipeline.

The Ontological Feedback Loop. The relationship between SGF and the upper ontology does not end at the export boundary. Because SGF captures novel terms dynamically, grounds them against NSM primes, and promotes them through the Ghost Protocol lifecycle, it can surface candidates for new categories that the upper ontology does not yet recognize.

The process:

1. A novel term enters through the intake pipeline and is captured as GHOST.
2. It accumulates evidence across documents and projects, moving to CUSTOM and then to CORE.
3. At CORE status, the term is a stable, grounded concept with verified IS_A chains.
4. The concept can be proposed as a candidate for a new CCO module or BFO extension.

This means SGF acts as a **discovery engine** for the upper ontology. It does not wait for the ontology to define categories before it can operate. It operates, discovers what exists in the language, and feeds back verified candidates for institutional adoption.

This means SGF does not just feed the upper ontology — it heals it. Upper ontologies ossify because their maintenance cycles cannot keep pace with human language. SGF inverts this: the ontology evolves bottom-up, in real time, wherever language operates. Standards bodies become ratifiers of emergent reality rather than speculative architects of it. The maintenance problem that has haunted ontology deployment since the field's inception is solved by giving the ontology a living intake mechanism that evolves alongside the language it processes. The ontology no longer has to be built before the system runs. It evolves alongside the language it processes.

2.6 Conclusion of Foundations

The Structural Necessity

Consider the alternative. A system that uses only BFO guarantees formal consistency but cannot prevent hallucination. The LLM fabricates a term. BFO classifies it. The system accepts garbage with mathematical certainty.

A system that uses only NSM prevents hallucination but cannot interoperate with institutional standards. The NSM core is brilliant linguistics. It is unknown in the DoD, in defense contracting, and in enterprise architecture. No review board will accept output that does not speak CCO.

There is no third option. You cannot fold cognitive grounding into BFO without breaking BFO's axioms. BFO requires that every entity be either independent, dependent, or occurrent. Cognitive primes like THINK and MAYBE do not fit. You cannot fold ontological compliance into NSM without losing NSM's universality. NSM primes are supposed to be universal across human languages. BFO's artifacts and processes are not.

The dual-axis model is not a design preference. It is a structural necessity. Meaning and existence are different things, modeled by different frameworks, serving different purposes. Any architecture that claims to provide both hallucination resistance and institutional compliance must separate them. There is no single-axis solution.

A Note on Philosophical Alignment

Some readers may recognize that this architecture bridges two traditions that have historically been treated as incompatible: the realist tradition that insists ontology must model only what exists independently of human cognition, and the conceptualist tradition that insists meaning is anchored in universal human concepts. This paper does not take sides in

that debate. It observes that both traditions describe something real about the world — and that any deployed system must operate in both domains simultaneously. The internal runtime must be able to process human language, which operates in the conceptualist domain. The export boundary must produce output that satisfies realist compliance standards. SGF does not resolve the philosophical debate. It renders it irrelevant by providing both layers, at different scales, connected by a deterministic bridge.

Neither replaces the other. Both are necessary. That is the point.

Key Principles Established in the Foundations

Principle	Meaning
Finite Bedrock Principle	Every unbounded domain needs a finite floor. 65 NSM primes terminate recursion.
Dual-Axis Necessity	Meaning and existence are different things, modeled by different frameworks. No single-axis solution exists.
Conservation Law of Meaning Transfer	Every crossing of meaning across a boundary requires pivot, bridge, policy, and proof.
BFO ensures consistency; NSM ensures truth	Neither alone is sufficient. Together they provide what no single framework can deliver.
The ontological feedback loop	The ontology evolves bottom-up, in real time, language by language. Standards bodies become ratifiers of emergent reality.

Meaning Infrastructure (Layers 1-3)

The Bridge from Foundations

The dual-axis architecture described in the Foundations section separates cognitive grounding (the semantic axis) from institutional compliance (the ontological axis). The seven layers described below implement this separation. Layer 1 provides the grounding floor for the semantic axis. Layers 2-3 provide the vocabulary and ingestion machinery for the

semantic axis. Layers 5-7 implement the export boundary and governance for the ontological axis. Layer 4 bridges them.

The reader should keep this dual-axis model in mind. Each layer serves one axis, the other, or the bridge between them.

The Stack Overview

The architecture has seven layers organized into two domains. Every layer solves a specific failure mode.

Domain 1: Meaning Infrastructure — what meaning is, how it is represented, how it is compiled.

Layer	Name	What It Solves
1	First Principles (Substrate)	Infinite regress, probabilistic grounding
2	Representation (Synapse + Lexicon)	Granularity mismatch, fragmented events
3	Engines (GLEAN, DB Adapter, Semantic CPU)	LLM hallucination, untraceable provenance

Domain 2: Communication and Governance — how meaning moves, what acts are performed, what actions are permitted.

Layer	Name	What It Solves
4	Discovery	How to find participants and capabilities
5	Transport (HFF)	Spoofing, replay, semantic corruption
6	Acts (AFP)	Ambiguous intent, ungoverned commitments
7	Governance (Omega)	No structural refusal, unauditable safety rules

Each layer is decoupled. You can adopt Layer 1 alone as a shared dictionary hub. You can add Layers 5-6 later for M2M communication. You can add Layer 7 later for governance. You can add the Wisdom Harvesting pipeline when your corpus reaches scale.

This section describes the first three layers — the meaning infrastructure.

Layer 1: First Principles — The Substrate

The problem it solves

Every definition depends on another definition. The chain never terminates. Systems eventually guess — they stop at the closest available definition and assume it is correct. This is the infinite regress problem, and it is the root cause of every probabilistic meaning failure.

The architectural answer

As established earlier, the **Finite Bedrock Principle** requires that every IS_A chain terminate at an NSM prime. The **Prime Registry** — 65 NSM semantic primes — is the implementation of this principle. Readers should refer back for the full description of NSM Ground Zero, the anti-hallucination shield, and the export bridge to BFO/CCO categories.

The following additional substrate principles are specified at this layer:

- **The Conservation Law of Meaning Transfer.** Every crossing of meaning across a boundary requires exactly four components: pivot (shared reference), bridge (verifiable connection), policy (rules for admission), and proof (record of the crossing). The failure modes are diagnostic: no pivot → grounding collapse, no bridge → isolation, no policy → anarchy, no proof → amnesia.
- **The clause as the atomic unit of meaning.** Triples fragment events; paragraphs are too coarse. The clause is the natural grain at which humans express events, and it is the natural grain at which machines should represent them.
- **15 thematic roles as a closed set.** The finite skeleton that every event in every domain maps onto. No new roles will ever be added. This is what makes integration cost N rather than N^2.
- **7 frames for classifying interpretation.** Act frame, normative frame, epistemic frame, experiential frame, constitutive frame, classificatory frame, evaluative frame. The frame tells the system how to interpret the claim — is it a command, a definition, an observation, a judgment?

Wisdom Harvesting connection

The Prime Registry itself does not learn. That is the point. It is the stable bedrock against which all learning is measured. However, if the Wisdom Harvesting pipeline is deployed, the *usage* of the registry — which primes are most frequently reached, which IS_A chains are

most commonly traversed — is harvested. This tells the system where the lexicon is most active and where expansion would have the highest impact.

Layer 2: Representation — The Synapse and the Lexicon

The problem it solves

In open predicate systems (RDF, OWL), every domain defines different relationship types. Aligning them requires N² mappings — every pair of systems must negotiate the meaning of their respective predicates. And triples are too fine-grained for natural claims: an event that involves an agent, a patient, a time, a location, an instrument, and a reason fragments across multiple triples, losing the structural integrity of the original event.

The architectural answer

Two data models coexist, each handling what the other cannot.

The Synapse is the event representation. Every clause compiles into exactly one Synapse: a verb hub with up to 15 fixed thematic roles.

```
HAS_AGENT, HAS_PATIENT, HAS_THEME, HAS_EXPERIENCER, HAS_RECIPIENT,
HAS_BENEFICIARY, HAS_TIME, HAS_LOCATION, HAS_SOURCE, HAS_DESTINATION,
HAS_MANNER, HAS_INSTRUMENT, HAS_CAUSE, HAS_REASON, HAS_ATTRIBUTE
```

No new roles will ever be added. Why close them? Because with closed roles, every event in every domain maps to the same skeleton. Integration cost drops from N² to N. An engineer does not need to learn a new relationship vocabulary for each domain — the same 15 roles serve contracts, medical guidelines, military operation orders, and manufacturing specifications.

The Synapse carries epistemic status (7 tiers, from CORE_DEFINITION through GHOST), provenance (source document, section, sentence, offset), rhetorical mode, verb features (mood, tense, aspect, negation, modality), and frame. Not all Synapses are equally authoritative. The system knows the difference between a definition and a rumor.

The 7 binary relations form the ontology axis:

```
IS_A, HAS_PART, HAS_MEMBER, HAS_INSTANCE, SAME_AS,
BROADER_THAN, NARROWER_THAN
```

These capture what exists and how things are categorized. They are separate from the event axis because ontology and events serve different purposes. An entity can be categorized

without being involved in any event. An event involves entities that are already categorized. Both axes are needed. Neither replaces the other.

Synapses compose into larger structures. A clause is atomic. But a recipe, a contract, a military operation order, or a diagnostic protocol is molecular — a chain of ordered steps, a nest of obligations, a lattice of evidence. Eight link types (CAUSES, SUPPORTS, CONTRADICTS, PRECEDES, DEPENDS_ON, ENABLES, DISABLES, SUBSUMES) compose Synapses into named SynapseGroups. The architecture is not flat. Molecular structures carry the same epistemic status, provenance, and governance guarantees as atomic ones.

The noun is a frozen verb

A "screwdriver" is not a static object. It is the verb DRIVE / SCREW frozen into a noun, carrying its functional role as a disposition. The same 15-slot grammar handles both the event (DRIVE) and the entity (SCREWDRIVER). The Y-axis (IS_A) tells you what it is. The X-axis (VerbHub + roles) tells you what it does. Both are needed.

The Five-Zone Lexicon

Every concept has a **Canonical ID** — the invariant address for meaning:

```
{language}.{lemma}.{microgloss}.{pos}.{namespace}
```

The first component is always the ISO language code, making the ID inherently multilingual. The microgloss disambiguates the sense. The namespace declares provenance.

Zone	Has Canonical ID?	TTL	Example
Core	✅ Yes	Permanent	`en.bank.financial_institution.noun.synapedia_wordnet`
Inferred	✅ Yes	Permanent after promotion	`en.titanium_torque_wrench.tool.noun.inferred`
Custom	✅ Yes	Scoped to document	`en.phils_diner.restaurant.noun.custom_20260512`
Instance	❌ No	Document lifetime	`inst.doc_20260512.a1b2c3d4.person`
Ghost	❌ No	30 days	`ghost.a1b2c3d4`

The Core zone is permanent and shared. The Custom zone is temporary and local. The Ghost zone is the system's way of saying "I know this exists, but I do not yet know what it is."

The three-axis theorem

Entity resolution across independently created descriptions requires three irreducible axes: ontology (Y), properties (P), and events (X). No two are sufficient. Any representation that lacks an event axis will fail on cases where the distinguishing information is carried by events.

Consider individuals who share the same name: a president, a general, an investment banker, a philanthropist. They share the same IS_A paths. The Y-axis and P-axis cannot distinguish them. The X-axis (events) can. One charged up a hill during a war. One landed on a beach during an invasion. One chaired a bank. One funded museums. If your representation lacks an event axis, you will confuse them. SGF does not confuse them, because the Synapse captures the event that distinguishes each.

Wisdom Harvesting connection

Every alignment attempt — whether it succeeds or fails — is harvested, if the Wisdom Harvesting pipeline is deployed. When a match succeeds, the IS_A chain and role bindings that produced the match are recorded as a reusable pattern. When a match fails, the GapReport names exactly where the chain broke. Over time, the system accumulates a library of successful and failed alignment patterns. Future alignments use this library to prioritize the most productive chain paths and avoid known dead ends.

Layer 3: Engines — GLEAN, the DB Adapter, and the Semantic CPU

The problem it solves

LLMs can generate plausible text, but they cannot verify that their output corresponds to anything real. They hallucinate specifications, invent citations, and fabricate data — all with perfect confidence. This is not a bug; it is an architectural consequence of predicting tokens rather than grounding meaning.

At the same time, most real-world knowledge exists in unstructured forms: prose documents, database records, regulatory texts, conversation transcripts. An architecture that cannot ingest these forms cannot reach the knowledge that organizations actually have.

The architectural answer

Three engines handle ingestion, transformation, and query. All three are governed by the same principle: **the LLM proposes; deterministic gates dispose.**

GLEAN is the prose-to-graph compiler. It takes natural language documents and produces Synapses. The pipeline is multi-stage and deterministic at every critical gate:

1. **Defluffer** — removes filler, hedging, redundancy before parsing.
2. **Entity Census** (7 passes) — NER, alias clustering, pronoun resolution, possessive chains, 12 metonymic patterns, context harvest, lexicon lookup.
3. **Participant Test** — a mention that appears in only one event is discarded. Only entities that participate in multiple events become permanent nodes. This prevents spurious entities from bloating the graph.
4. **Coverage Gate** — if more than 2% of the vocabulary in the input document is not found in any zone of the lexicon, the pipeline halts. Silence is better than confident nonsense.
5. **Reconstruction Test** — a second, independent LLM receives the extracted Synapses and regenerates a claim-level summary. The summary is compared to the original document — not for stylistic fidelity, but for propositional parity. If 100 input facts produce 100 output citations, the extraction preserves information. Omissions signal Information Decay. Additions signal Information Inflation. Both constitute failures.
6. **Triangulated axiom validation** — three independent checks: self-consistency, cross-validation against existing Synapses, and rule-based logical checks.

When a mention cannot be resolved, the system mints a **Ghost** — a provisional node with a 30-day TTL. It carries UNKNOWN epistemic status and is never available for reasoning. Ghosts that accumulate evidence across multiple documents are promoted to full minted entries. Ghosts that do not meet the threshold within 30 days are garbage-collected.

The DB Adapter transforms structured data — database schemas, SQL queries, CSV exports — into Synapses. It performs the same function as GLEAN for data sources. The output is a Synapse graph that can be queried alongside prose-extracted knowledge.

The Semantic CPU is the query and reasoning interface. Given a query expressed as a Synapse pattern, it walks the graph, follows roles and links, applies epistemic status filters, checks Omega rules when governance is present, and returns results with full provenance. It does not predict tokens. It resolves canonical IDs, follows IS_A chains, traverses role bindings, and returns what it finds. If the structure does not contain the answer, it returns a GapReport explaining exactly what is missing.

Dimension	LLM	Semantic CPU
Source citation	None, or hallucinated	Specific, with document and page

Dimension	LLM	Semantic CPU
Verifiability	Impossible	Full ProofTrace
Currentness	Training cutoff	Knowledge Pack version
Update cost	Millions (retraining)	A file download
User trust	"It sounds right"	"I can check the source"

Wisdom Harvesting connection

If the Wisdom Harvesting pipeline is deployed, every GapReport is harvested. The system learns which question patterns consistently produce UNKNOWN and proactively generates Knowledge Pack expansion requests for those areas. Every successful query strengthens the rule corpus for similar queries. Over time, the Semantic CPU returns fewer GapReports and more ProofTraces without any change to the underlying model — because the knowledge base has expanded.

Summary

The first three layers establish the **meaning infrastructure**:

Layer	What It Provides	Key Innovation
1: First Principles	Finite bedrock for meaning	65 NSM primes terminate IS_A chains
2: Representation	Atomic event structure and shared lexicon	Synapse (verb hub + 15 closed roles) + five-zone Canonical ID system
3: Engines	Deterministic ingestion and query	LLM proposes; deterministic gates dispose

Without these layers, every system eventually guesses, events fragment across triples, and hallucinated facts enter the knowledge base unchecked. With these layers, meaning has a finite floor, a clause-grain atom, and a deterministic ingestion pipeline.

Communication and Discovery (Layers 4-6)

The Bridge from Meaning Infrastructure

The previous section established the meaning infrastructure: a finite bedrock of NSM primes (Layer 1), a clause-grain Synapse representation with a shared lexicon (Layer 2), and deterministic engines that ingest prose and structured data into the Synapse graph (Layer 3).

With these layers, a machine can *represent* meaning. But it cannot yet *exchange* meaning with another machine — because exchanging meaning requires finding the other machine, agreeing on terms, verifying integrity, and declaring intent.

Layers 4-6 provide this capability. They sit in Domain 2 (Communication and Governance) and handle how meaning moves across trust boundaries.

Layer 4: Discovery — Who Is There and What Can They Do?

The problem it solves

Before two machines can exchange meaning, they must find each other and establish what each is capable of doing. Current discovery mechanisms (UDDI, DNS-SD, mDNS) handle service location but not capability alignment. They tell you that a service exists. They do not tell you what acts the service is authorized to perform, what lexicon it uses, or what governance rules constrain it.

In a coalition context, the problem is amplified. A ground station may encounter a drone from an allied nation that it has never communicated with before. The drone is capable of receiving commands, sharing sensor data, and coordinating maneuvers — but the ground station does not know which protocols, lexicons, or security frameworks the drone speaks. Without a structured discovery mechanism, the integration timeline is measured in months.

The architectural answer

Every SGF-compliant system publishes a **capability manifest** that declares its identity, capabilities, and constraints. The manifest is the basis for the **Stranger Rule** — a system that has never met another can establish the terms of communication on first contact.

Standard discovery endpoint. Participants that support HTTP-based discovery expose their capability manifest at a well-known location: `/.well-known/graph`. Other transports (V2X radio, satellite, tactical data links) define equivalent, profile-specific discovery locations. The manifest content remains the same logical structure regardless of transport. This makes integration predictable — no custom API documentation is needed per partner.

Required manifest fields:

- **participant_id** — a stable identifier that survives hardware changes. For mobile or privacy-sensitive participants, a `temporary_participant_id` may be used with an `expiry_window` and `replay_prevention` mechanism.
- **supported_sgf_versions** — which SGF Core versions the participant can understand.
- **supported_hff_versions** — which HFF protocol versions it can speak.
- **supported_afp_versions** — which AFP protocol versions it supports.
- **endpoints** — where to send HFF/AFP messages (URLs, radio channels, with protocol and profile hints).

Recommended manifest fields:

- **supported_encoding_profiles** — which HFF encoding profiles (JSON, CBOR, binary) the participant accepts.
- **supported_lexicons** — which Core and domain lexicon releases it can hydrate.
- **supported_knowledge_packs** — which Knowledge Packs it recognizes by ID and version.
- **capabilities** — functional roles (vehicle, drone, trading engine, medical device, agentic assistant).
- **trust_anchors** — CA roots, key registries, or authority registries the participant trusts.
- **auth_methods** — how the participant authenticates peers (mTLS, signed HFF, OAuth at the gateway).
- **rate_limits** — acceptable message rates for different classes of peers.
- **max_payload_size** — upper bounds on message size.
- **supported_act_types** — which AFP act types the participant will accept (accepts INFORM and REQUEST but not COMMAND).
- **supported_domain_profiles** — domain-specific profiles (automotive V2X, medical device, regulated transaction).

Broadcast scope and temporary identity. Mobile, short-lived, or privacy-sensitive systems may use:

- **recipient_scope** — which receivers a message targets (`vehicles_within_800m_ahead`, `drones_in_formation_alpha`, `robots_in_warehouse_zone_12`).
- **broadcast_scope** — a wider anonymous broadcast domain (all vehicles in a region).
- **temporary_participant_id** — a short-lived identifier instead of a permanent `participant_id`.
- **trust_anchor** and **credential_or_certificate** — allow receivers to validate the temporary participant.
- **expiry_window** and **replay_prevention** — bound how long the temporary identity remains valid.

Identity strength for high-risk operations. Messages that can cause high-risk physical, financial, or safety effects (HIGH_RISK_COMMAND or equivalent domain profiles) **MUST NOT**

be admitted based solely on a `temporary_participant_id` without a strong credential binding to a trusted anchor. Long-lived identities bound to attestations or authority registries are required for high-risk commands and regulated transactions. A drone with a temporary ID can receive emergency broadcast information but cannot issue a retargeting command unless it also presents a verifiable credential from a known trust anchor.

Trust evaluation is not automatic. Publishing a capability manifest does not entitle a participant to be trusted. Receivers **MUST** evaluate `trust_anchors`, credentials, and revocation status before treating a manifest as authoritative for any role. Implementations **SHOULD** treat mismatches between manifest claims (such as capabilities or roles) and observed behavior as grounds for downgrading or revoking trust.

Trust anchor to key binding. Capability manifests describe which trust anchors a participant uses. HFF messages carry concrete keys and signatures. The binding between them is structural: the manifest declares "I trust these anchors," and each HFF message carries a key whose certificate chains to one of those anchors. If a message arrives with a key that does not chain to a declared trust anchor, the manifest claim and the message credential are inconsistent, and the message must be rejected. This binding ensures that manifest declarations are not purely advisory — they are cryptographically verifiable.

In adversarial scenarios — prank emergency broadcasters, hostile swarm nodes — receivers rely on Discovery manifests plus HFF security to refuse acts from participants whose manifests and keys are not trusted for the claimed roles.

The Discovery flow

Discovery is the prelude to SGF communication. A typical flow:

1. A receiver fetches or receives a capability manifest (from `/.well-known/graph` or the appropriate transport-specific location).
2. It checks `participant_id`, supported versions, encoding profiles, lexicons, and `trust_anchors`.
3. It decides whether this participant is eligible to communicate under local policy.
4. If eligible, it uses `endpoints` and the declared profiles to establish HFF/AFP exchange.

Discovery does not override receiver sovereignty. A participant may publish a manifest; other participants remain free to ignore it.

Wisdom Harvesting connection

If the Wisdom Harvesting pipeline is deployed, each first-contact discovery interaction produces a partner profile Knowledge Pack — the partner's typical lexicon coverage, common frame patterns, and authority constraints. The 10th new partner integrates faster than the 1st

because the system has learned which mismatches to expect and which negotiation patterns are most efficient.

Layer 5: Transport — HFF (Honest Fact Forwarding)

The problem it solves

When a message crosses a trust boundary — between two machines, two organizations, two classification domains — the receiver must verify that the message is authentic, unmodified, current, and intended for them. Current transport protocols (HTTP, MQTT, AMQP) handle delivery but not semantic integrity. They guarantee that bytes arrived. They do not guarantee that the meaning encoded in those bytes survived transport without corruption.

For a weapon system, the difference is existential. A signed message guarantees who sent it. It does not guarantee that the command embedded in that message is the command the sender intended — an adversary can tamper with the meaning while leaving the signature intact. HFF closes this gap by binding semantic integrity to cryptographic integrity.

The architectural answer

HFF provides the transport envelope for all SGF communication. Every HFF message carries a set of fields that together answer five distinct security questions.

Required envelope fields:

- **hff_version** — identifies the HFF spec version.
- **encoding_profile** — declares the actual byte encoding (canonical JSON, CBOR, binary).
- **message_id** — a unique identifier for replay detection and receipts.
- **created_at** — when the message was formed.
- **sender** — identifies the sending participant.
- **payload** — the SGF objects (Synapses, lexicon entries, governance rules) being exchanged.
- **integrity** — carries hashes, signatures, and related security metadata.

Recommended envelope fields:

- **recipient_ref** or **recipient_scope** — addresses a specific receiver or a class of receivers.
- **expires_at** — bounds freshness; messages past expiry must not be admitted.
- **nonce** — prevents replay within the validity window.
- **conversation_id** — ties related messages, especially when AFP is in use.
- **core_lexicon_release** — identifies the Core Lexicon version used to ground terms.
- **trust_anchor_ref** — points at the trust root the receiver should use to evaluate keys.

The signing rule. The sender signs the canonical bytes of the entire HFF logical message under the declared canonicalization and encoding profile. This means the envelope, headers, payload, and all metadata are covered by the signature — not just the payload content alone. A receiver verifies the signature against the canonical bytes before examining any field. This prevents signature stripping attacks, where an adversary removes a payload from its authentic envelope and re-wraps it in a different context. The signature binds the payload to its specific envelope and sender irrevocably.

The five gates. Before admitting any HFF message, a receiver answers five questions in order:

Gate	Question	What It Prevents
Schema	Does the message conform to the HFF envelope format?	Malformed or truncated messages
Hash	Has the content changed since it was signed?	Tampering, corruption
Signature	Is the sender who they claim to be?	Impersonation
Freshness	Is this message current?	Replay attacks
Hydration	Can all terms be resolved against the lexicon?	Semantic drift, unknown vocabulary

A message that fails any gate must be rejected or quarantined according to the security profile. A signed, fresh, well-formed message is a candidate for admission — not an instruction to obey. Local governance (Layer 7) determines what happens next.

The full receiver decision pipeline. Beyond the five gates, HFF defines an ordered eight-step pipeline that every HFF-compliant system must execute before admitting a message from an unknown or untrusted sender:

1. **Schema validation** — Is the message well-formed under the declared schema and HFF version?
2. **Hash verification** — Do the payload hash and content hash match the actual content?
3. **Signature verification** — Is the signature valid under a trusted key? Is the key itself valid (not revoked, not expired)?
4. **Freshness check** — Is the message current (expiry and replay cache check)?
5. **Lexicon hydration** — Can all terms be resolved from known lexicons and manifests?
6. **Authority evaluation** — Is the sender authorized for the requested act and risk class?
7. **Local policy check** — Does local policy allow this action?

8. **Safety profile evaluation** — Does the safety profile pass given current mission and world state?

A message that fails any required step must be rejected or quarantined. A message that passes all eight steps is admitted to the reasoning graph — but it is still not obeyed until Omega governance (Layer 7) produces an ALLOW verdict.

Communication patterns. HFF supports all common communication patterns. Security profiles constrain how these patterns are used:

- **1:1** — point-to-point command and control (ground station to specific drone). Uses `recipient_ref`.
- **1:M** — broadcast to multiple receivers (emergency alert to all vehicles in an area). Uses `recipient_scope`.
- **M:1** — multiple senders to a single receiver (sensor fusion inputs to a command node). Uses `recipient_ref`.
- **M:M** — swarm coordination (multiple drones exchanging position data). Uses `recipient_scope` with group keys.

The admission pipeline — integrity, authenticity, authorization, freshness, confidentiality, hydration — applies identically regardless of party count. Only the profiles and local policy differ.

Security profiles. HFF defines reusable security profiles that specify which gates are mandatory for a given class of communication. The profile decides which questions must be answered at the wire; local policy can always demand more.

- **PUBLIC_SIGNED_BROADCAST** — public, integrity-sensitive broadcasts. Requires signature, trust_anchor, expiry, nonce, and payload hash. Encryption is not used; anti-spoofing comes from verification, not secrecy. Used for emergency vehicle advisories, hazard warnings, public road alerts.
- **CONFIDENTIAL_DIRECT** — specific recipients only. Requires encryption envelope, signature, trust_anchor, expiry, and nonce. Only authorized recipients can read the payload. Used for financial, legal, medical, commercial, and military messages.
- **CONFIDENTIAL_GROUP** — addressed to a defined group (swarm, fleet, team). Requires encryption envelope, group key reference, signature, trust_anchor, expiry, and nonce. Group membership, key rotation, and revocation are handled by the deployment's trust infrastructure.
- **HIGH_RISK_COMMAND** — commands that control vehicles, weapons, drones, robots, medical devices, money, or critical infrastructure. Requires authority_frame_id, risk_class, safety_profile, ack_required, receipt_policy, replay_window, and revocation_check. Encryption is recommended unless signed public command broadcast is explicitly required by the domain profile.

High-risk receivers must not act merely because a message is authentic. They must also validate authority, local policy, safety constraints, mission state, and current world state before obeying.

Multi-act payloads. A single HFF message may carry multiple AFP acts that share a security envelope and transport cost. When an HFF message carries multiple acts, the payload includes `acts[]` where each act entry includes `act_id`, `illocution`, `payload_ref`, `ack_required`, `deadline`, and `authority_required`. This allows a sender to bundle related acts — INFORM + REQUEST + ADVISE in an emergency broadcast — while keeping each act individually addressable and auditable. The security envelope covers all acts in the bundle; individual acts do not require separate HFF messages.

Encryption envelope. When confidentiality is required, HFF uses an encryption envelope within the message structure. Recommended fields:

- **encryption_profile** — identifies the cryptographic suite and mode.
- **encrypted_payload_ref** — points to the encrypted bytes.
- **content_key_ref** and **recipient_key_refs** — describe how content keys are distributed.
- **encrypted_fields** and **unencrypted_headers** — clarify which parts remain visible for routing and profile selection.

No proprietary design data leaves the air gap. The encryption envelope ensures that even if the transport layer is compromised, the payload remains confidential to the intended recipients.

Cryptographic profiles and downgrade resistance. HFF does not mandate a single global cryptographic suite, but high-risk and regulated deployments MUST adopt a declared cryptographic profile and enforce it consistently. Implementations **MUST NOT** use deprecated or known-weak algorithms or key sizes in any profile that carries HIGH_RISK_COMMAND or CONFIDENTIAL messages. Implementations **SHOULD** define one or more named cryptographic profiles (e.g., `HFF_CRYPTO_PROFILE_1`) that specify key types, minimum key sizes, signature algorithms, hash algorithms, and encryption algorithms.

Downgrade resistance. When a sender and receiver have previously communicated using a given combination of schema version, SGF Core version, and Core Lexicon release under a particular trust anchor, receivers **SHOULD NOT** silently accept messages from the same sender key that claim materially older schema or lexicon versions without an explicit local-policy decision. This prevents downgrade attacks where an adversary forces the protocol to fall back to a weaker cryptographic regime or a known-vulnerable schema version.

Late and stale message handling. `expires_at` and any profile-specific `replay_window` define a validity window. A message whose `expires_at` is in the past **MUST** be rejected or quarantined, even if it was originally part of a valid conversation. Late ACK, CONFIRM, or ERROR messages received after expiry **MAY** be logged for audit but **MUST NOT** trigger renewed execution or

state changes. In particular, stale PUBLIC_SIGNED_BROADCAST and HIGH_RISK_COMMAND messages must not be obeyed simply because they are freshly replayed and have valid signatures.

The Stranger Rule. Two machines that have never met can communicate on first contact, because both tether to the shared Core Lexicon and the sender brings its own lexicon entries for non-core terms. The receiver follows the tethering links back to shared ground and resolves the meaning. No prior agreement about the specific terms was needed.

This rule is what makes coalition integration possible at the timeline that operational commanders demand. A ground station and an allied drone that have never communicated can exchange mission parameters, governance constraints, and ontological commitments on first contact. The integration happens in seconds, not months.

Wisdom Harvesting connection

If the Wisdom Harvesting pipeline is deployed, each first-contact integration produces a partner profile Knowledge Pack — the partner's typical lexicon coverage, common frame patterns, and authority constraints. The 10th new partner integrates faster than the 1st because the system has learned which mismatches to expect and which negotiation patterns are most efficient.

Layer 6: Acts — AFP (Act Framing Protocol)

The problem it solves

A message that says "The pump meets MIL-STD-810H" could be an INFORM (the pump is certified), a PROMISE (we will deliver a certified pump), or a COMMAND (ensure the pump meets this standard). The literal text does not distinguish them. The receiver must interpret which act is being performed — and the wrong interpretation leads to the wrong action.

In coalition operations, the problem is compounded by different command cultures. What one nation treats as a COMMAND (obligation to execute) another treats as an ADVISE (recommendation, not binding). AFP eliminates this ambiguity by declaring the act explicitly.

The architectural answer

AFP is the act and conversation layer that sits on top of HFF. HFF moves meaning. AFP *acts* with meaning. AFP declares what each message is doing — informing, requesting, commanding, promising, accepting, refusing — so the receiver knows what is being asked and what authority is required.

Single-act envelope. A single AFP message is carried inside an HFF payload. Required fields:

- **afp_version** — identifies the AFP spec version.
- **afp_message_id** — identifier for this act-level message within a thread.
- **thread_id** — ties this act into an ongoing conversation or transaction.
- **sender_id** — identifies the sender at the act layer (aligned with HFF sender).
- **illocution** — one of the AFP act types.
- **payload_ref** — points to the SGF payload (Synapse, group, or bundle) that this act concerns.
- **hff_payload** — the enclosing HFF message or a reference to it.
- **security_envelope** — points to or embeds the HFF integrity and security fields.

Multi-act envelope. A single HFF message may carry multiple AFP acts that share a security envelope and transport cost. Each act in the bundle includes `act_id`, `illocution`, `conversation_transition`, `payload_ref`, `authority_required`, `ack_required`, and `deadline`. This allows a sender to bundle related acts — INFORM + REQUEST + ADVISE in an emergency broadcast — while keeping each act addressable and auditable. The security envelope covers all acts in the bundle.

Act types. AFP defines a closed set of 13 act types for v1.0:

Act Type	Meaning
INFORM	The sender asserts a claim about the world
ADVISE	The sender recommends a course of action
REQUEST	The sender asks the receiver to perform an action
QUERY	The sender asks the receiver for information
COMMAND	The sender directs the receiver to perform an action
PROMISE	The sender commits to a future action
PROPOSE	The sender offers a deal or exchange
ACCEPT	The sender agrees to a proposal
REFUSE	The sender rejects a proposal
CANCEL	The sender withdraws a prior commitment
CONFIRM	The sender asserts that a prior claim is still valid

Act Type	Meaning
ACK	The sender acknowledges receipt
ERROR	The sender reports a protocol or validation failure

These are illocution labels, not SGF roles. They distinguish delivery from agreement, agreement from execution, and refusal from technical error.

Conversation transitions. Act types describe *what* is being done. Conversation transition labels describe *how the conversation state changes*. AFP defines the following transitions:

Transition	Meaning
PROPOSE	A proposal is made
COUNTER	A counter-proposal is made
ACCEPT	A proposal is accepted
EXECUTE	An action is executed
CONFIRM	Execution is confirmed
REFUSE	A proposal is rejected
ESCALATE	A decision is escalated to a higher authority
CANCEL	A prior act is withdrawn
EXPIRE	A deadline has passed with no response
ERROR	A protocol or validation failure occurred

Act types and conversation transitions are related but not identical. A PROPOSE act usually carries `conversation_transition = PROPOSE`. An ACCEPT act carries `conversation_transition = ACCEPT`. A COMMAND act might carry `conversation_transition = EXECUTE`. A REFUSE act carries `conversation_transition = REFUSE`.

Illegal transitions — for example, EXECUTE without any prior ACCEPT when a profile requires it, or CANCEL of an act that was already executed — must return a structured ERROR act that explains which rule was violated. This ensures that machine-to-machine coordination is not

ambiguous: every party knows what state the conversation is in, and no act can be silently ignored.

Error codes. When an ERROR act is emitted, it carries a structured error reason drawn from a defined taxonomy. Examples include:

- `PROTOCOL_ILLEGAL_TRANSITION` — the attempted conversation transition is not permitted from the current state.
- `PROTOCOL_MISSING_AUTHORITY` — the act requires authority that was not provided.
- `PROTOCOL_EXPIRED` — the act arrived after its deadline.
- `HFF_SIGNATURE_INVALID` — the enclosing HFF message has an invalid signature.
- `HFF_EXPIRED` — the enclosing HFF message is past its expiry.
- `HFF_REPLAY_DETECTED` — the enclosing HFF message was previously received.
- `HFF_MISSING_LEXICON` — a term in the payload cannot be hydrated.

Domain profiles may refine the error taxonomy, but they must not reinterpret an HFF-layer failure as successful ACCEPT or CONFIRM.

Deadlines, expiry, and incomplete conversations. The `deadline` field at the act level and HFF `expires_at` together define a validity window for an act. If no ACCEPT, REFUSE, CONFIRM, or ERROR is received before the deadline (or `expires_at`, whichever is stricter), the sender **MAY** emit an EXPIRE act and **MUST** treat the original act as no longer actionable. Late ACCEPT or CONFIRM received after the validity window **MUST NOT** resurrect an expired or cancelled act. An implementation **MAY** treat such late responses as new proposals or simply REFUSE.

Domain profiles with long communication delays (for example, deep-space probes) must choose deadlines and expiry windows appropriate to their latency, but those windows are still hard bounds for action. Participants **MUST NOT** assume success from silence — absence of CONFIRM means there is no protocol guarantee that the act was executed. Implementations **SHOULD** track conversation state and emit ERROR or EXPIRE when a required response is missing or an illegal transition occurs.

Authority is act-specific. A sender may be authorized to INFORM but not to COMMAND, or to command only within specific domains or jurisdictions. The capability manifest (Layer 4) and the AuthorityFrame (carried in the act's security envelope) together determine whether an authenticated sender is allowed to issue a specific act in a specific context. Signature alone is never sufficient for obedience.

COMMAND, CANCEL, high-risk REQUEST, and any act with real-world effect must be validated against an AuthorityFrame before execution. The AuthorityFrame, interpreted under local policy, decides whether an authenticated sender is authorized to issue a specific act in a specific context.

The Pass the Salt Principle. "Can you pass the salt?" is not an interrogative about capability. It is a request. The Double-Entry Ledger preserves both the literal surface form and the interpreted intent. The Divergence Score — the distance between the surface form embedding and the interpreted event embedding — serves as a security primitive. A high Divergence Score on a safety-critical utterance triggers a confirmation interrupt: the system asks for human confirmation before acting.

Binding AFP to HFF security. AFP acts are always evaluated under the security context of the enclosing HFF message. Each AFP act's `security_envelope` **MUST** correspond to exactly one HFF message whose security profile is appropriate for the act's illocution and risk class. If HFF validation fails for a message — schema error, invalid signature, untrusted `trust_anchor_ref`, expiry, replay, missing required lexicon — the AFP layer **MUST NOT** treat any contained acts as valid inputs to a reasoning graph or actuator. An HFF-layer failure cannot be overridden by AFP-layer processing.

When HFF validation fails, implementations **SHOULD** emit an AFP ERROR act with a structured error reason that indicates the failure class: `HFF_SIGNATURE_INVALID`, `HFF_EXPIRED`, `HFF_REPLAY_DETECTED`, `HFF_UNTRUSTED_ANCHOR`, `HFF_MISSING_LEXICON`, or `HFF_UNSUPPORTED_PROFILE`. These error codes ensure that the sender knows exactly why the message was rejected and can take corrective action.

Receipts and acknowledgments. AFP distinguishes several receipt-related acts:

- **ACK** — confirms receipt and parsing. Does not indicate agreement or execution.
- **CONFIRM** — records that an act was completed or that a claimed state has been verified.
- **REFUSE** — indicates that an act or proposal was rejected.
- **ERROR** — reports protocol failure, invalid transition, missing lexicon, missing authority, validation failure, or unsafe action.

This separation lets systems distinguish delivery from agreement, agreement from successful execution, and refusal from technical error. A weapon system that receives an ACK knows the command arrived. It does not know the command was obeyed until it receives a CONFIRM.

The Decider

The Decider processes alignment results and produces one of five outcomes:

Outcome	Meaning
ACCEPT	Verified. Execute or record.

Outcome	Meaning
REJECT	Invalid. Produce GapReport explaining why.
CONDITIONAL	Valid under specified constraints. Execute only within the constraints.
CLARIFY	Moderate confidence. One round trip to the sender for disambiguation.
ESCALATE	Too risky or uncertain. Human must decide.

The system never guesses. When it does not know, it says so. The Decider applies human-authored policy (Stream of Consciousness) to determine the outcome.

Wisdom Harvesting connection

If the Wisdom Harvesting pipeline is deployed, each dispute — a REFUSE that was unexpected, a CLARIFY that took multiple rounds, an ESCALATE that could have been handled autonomously — is harvested. The system learns which conversation patterns lead to deadlock and generates preemptive rules to avoid similar disputes in the future. Over time, the CLARIFY and ESCALATE rates drop as the system learns the common patterns.

Summary

Layers 4-6 establish the **communication infrastructure**:

Layer	What It Provides	Key Innovation
4: Discovery	How to find participants and their capabilities	Capability manifests with trust anchors, the Stranger Rule
5: Transport (HFF)	How to move meaning across trust boundaries	Five admission gates, eight-step pipeline, security profiles
6: Acts (AFP)	How to declare what act is being performed	13 illocution types, conversation state, the Decider

Without these layers, every integration is a bespoke project, meaning can be corrupted in transit without detection, and the receiver must guess the sender's intent. With these layers, two machines that have never met can find each other, exchange meaning with integrity guarantees, and know exactly what act is being performed.

Governance and the Stack as a Whole (Layer 7 + Summary)

The Bridge from Communication Layers

The previous section established the communication infrastructure: how machines find each other (Layer 4), how meaning moves across trust boundaries (Layer 5), and how acts are declared (Layer 6). With these layers, a machine can discover a peer, receive a message with integrity guarantees, and know what act is being performed.

But knowing what a message *means* is not the same as knowing whether the requested action is *permitted*. A system that can parse a command but cannot refuse it is a system that can be compelled to do anything. Governance cannot be probabilistic — a safety rule that is "usually" enforced is not a safety rule.

Layer 7 provides this capability. It sits at the top of the stack, governing everything below it.

Layer 7: Governance — Omega

The problem it solves

Every previous layer answers "what does this message mean?" and "what act is being performed?" None answers "is this action permitted?" Systems that cannot refuse commands are systems that can be compelled to do anything. Governance cannot be probabilistic — a safety rule that is "usually" enforced is not a safety rule.

The problem is compounded by the fact that modern LLM-based systems have no structural mechanism for refusal. Guardrails are implemented as system prompts — overridable by a sufficiently persuasive user, by jailbreak techniques, or simply by the model's training to be helpful overcoming the guardrail when the user seems insistent. There is no mechanism for "no" that the user cannot override.

The architectural answer

Omega is a typed, non-Turing-complete governance language with a formally specified grammar, a safety kernel, and a constitutional amendment procedure. It sits between the probabilistic reasoning layer and the deterministic action layer, ensuring that no command reaches an actuator unless a compiled rule explicitly permits it.

The 13 atomic primitives are fixed at the Constitutional tier and cannot be altered by extension. They separate into two groups by what they operate on:

Nine object-primitives specify the system being modeled, its environment, and its data:

```
CONTEXT_RULE, TEMPORAL_RELATION, RESOURCE_BOUND,
ENVIRONMENT_INTERFACE_POINT, DATA_TYPE_SCHEMA, STATE_TRANSITION,
TRUST_ELEMENT, PERCEPTION_MAP, LEARNING_AXIOM
```

Four meta-primitives operate on rules and on the specification itself:

```
GOVERNANCE_RULE, SELF_REFERENCE_POINT, MUTATION_RULE,
META_DEFINITION_RULE
```

The split is structural, not notational. Object-primitives quantify over the modeled system. Meta-primitives quantify over rules, references, mutations, and definitions within the specification.

The CAN → MAY → DO gate is the structural core:

- **PERCEPTION_MAP** answers CAN — it maps external signals into typed claims about what the system can observe. The drone's PERCEPTION_MAP says: "I can observe my GPS coordinates, my battery level, my network connectivity status, and incoming command messages."
- **GOVERNANCE_RULE** answers MAY — it returns ALLOW, DENY, or UNKNOWN. "If a HIGH_RISK_COMMAND arrives during a period of degraded network connectivity and no human TRUST_ELEMENT is present, the action is DENY."
- **STATE_TRANSITION** is DO — it applies permitted changes when MAY returns ALLOW. If MAY is DENY or UNKNOWN, the STATE_TRANSITION is blocked. The command is refused. The GapReport names the missing authorization.

The sequence is enforced at load time, not runtime. A GOVERNANCE_RULE that requires predicates not supplied by any PERCEPTION_MAP is rejected before it ever runs. A STATE_TRANSITION not justified by at least one applicable GOVERNANCE_RULE with verdict ALLOW is invalid.

Two profiles:

Profile	What It Admits	Decidability
Strict	Boolean expressions, comparisons, set-membership tests	Statically decidable
Extended	Full pseudocode: IF, LOOP, FUNCTION, recursion	Bounded by RESOURCE_BOUND; UNKNOWN if exceeded

The Strict profile is used for safety-critical rules that must be verified at compile time. The Extended profile is used for complex policy evaluation with bounded recursion. Both are governed by the same Constitutional constraints.

Default profile. If no PROFILE directive appears in a module, the specification defaults to the Strict Profile. This means every Omega module is safety-critical by default — the author must explicitly choose to use the Extended Profile for modules that require bounded general computation.

Fail-closed: no matching rule → DENY. A command that does not match any GOVERNANCE_RULE cannot be executed. This is the opposite of most authorization systems, where no rule defaults to ALLOW. In safety-critical autonomous systems, the default must be DENY.

The Event Horizon. The architecture separates probabilistic reasoning from deterministic action. The Event Horizon is a deterministic membrane: probabilistic thought (LLM proposals, Synapse alignment, governance evaluation) happens above it. Physical action (actuators, network writes, file I/O) happens below it. The Kernel — small, deterministic, never-changing — mediates crossing.

For real-time robotic control, low-latency reflex rules live below the Event Horizon and execute deterministically. Omega rules for reflex actions compile to sub-millisecond evaluation. The safety kernel can execute within a hard real-time deadline — typically under 100 microseconds on embedded hardware, depending on rule complexity. The LLM cannot override these reflex rules — it can only propose changes to the Governance layer, which may update reflex rules after evaluation. This means a robot can react to a collision faster than an LLM can generate a token, while still being governed by the same constitutional constraints.

The ability to mint new laws. The Constitution is fixed. Its 13 primitives cannot be altered. But within the Constitution's constraints, new GOVERNANCE_RULEs can be minted. A drone that discovers a new threat pattern — a specific spoofing technique that bypasses the standard authentication framework — can generate a new GOVERNANCE_RULE that prevents similar threats. The Constitution ensures the new rule does not violate core safety constraints — it cannot contradict a Constitutional principle, it cannot exceed its RESOURCE_BOUND, and it cannot create a MUTATION_RULE that would alter the Constitution itself.

This is how the system learns to govern itself without becoming unpredictable. The kernel stays small and stable. The governance layer grows within constitutional constraints.

A note on MUTATION_RULE as a named composition. MUTATION_RULE can be reconstructed as a composition of SELF_REFERENCE_POINT (the target to modify), GOVERNANCE_RULE (the condition and approval policy), and STATE_TRANSITION (the transform action). It remains in the canonical set of 13 primitives because self-modification is so common, structurally rich, and conceptually cohesive that giving it a dedicated name

produces clearer specifications than forcing every author to compose the same three primitives by hand. The same principle applies to SQL's JOIN (expressible as cross product plus filter but named for readability) and to every mature specification language. The 13-primitive count is the residue at a level that balances irreducibility with ergonomics.

The evaluator model

Omega is executed by a **Safety Kernel** — a deterministic, non-probabilistic engine that sits between the probabilistic reasoning layer and the physical action layer. The Safety Kernel exposes three operations:

LOAD — parses a `.omega` specification against the canonical EBNF grammar. At load time, the evaluator performs the following static checks:

1. **Grammar parse** — the source must conform to the canonical EBNF. Every production rule must match. Errors carry line and column of the first non-conforming token.
2. **Reference resolution** — every named reference (RuleID, EntityID, ContextID, ScopeID, SchemaID, BoundID) must resolve to a definition within the loaded spec. Dangling references are load-time errors.
3. **Profile compliance** — for PROFILE Strict (or default Strict), no LOOP, WHILE, FOR, or FUNCTION may appear inside a predicate body, condition body, or transform-action body.
4. **Required-field presence** — each of the 13 primitives has a defined set of required fields. A call that omits a required field or includes a field not defined in the grammar is a load-time error.
5. **MUTATION_RULE topology** — the directed graph of MUTATION_RULE chains must be acyclic. A cycle (MUTATION_RULE A names MUTATION_RULE B and MUTATION_RULE B names MUTATION_RULE A, directly or transitively) is a load-time error.
6. **Type compatibility** — the types of values produced by PERCEPTION_MAP output schemas must be compatible with the types consumed by GOVERNANCE_RULE predicates that reference them.
7. **Cross-reference structural integrity** — every MUTATION_RULE's TARGET_REFERENCE must name a SELF_REFERENCE_POINT. Every GOVERNANCE_RULE's ENFORCEMENT_CONTEXT must name a CONTEXT_RULE. Every LEARNING_AXIOM'S CONSTRAINT_SET entries must each name a RESOURCE_BOUND.

A spec that passes all static checks produces a compiled spec object. A spec that fails any check produces a structured parse error — it is never silently accepted.

EVALUATE — takes a proposed action descriptor and a current state object, and produces a verdict. The evaluation proceeds as follows:

1. **Scope matching.** Identify the set of GOVERNANCE_RULEs in the loaded spec whose SCOPE matches the proposed action. If no rule's SCOPE matches, the default-deny

policy applies: the action is rejected with verdict DENY, rule_id NULL. Omega is fail-closed.

2. **Predicate evaluation.** For each matching GOVERNANCE_RULE, evaluate its PREDICATE against the current state object. Boolean sub-expressions are evaluated following the operational semantics of the grammar. In the Strict Profile, all predicate terms must resolve before a final verdict is issued.
3. **Verdict per rule.** If the predicate evaluates to TRUE and the ENFORCEMENT_CONTEXT is permissive, emit ALLOW with the rule_id. If the predicate evaluates to TRUE and the ENFORCEMENT_CONTEXT is prohibitive, emit DENY with a structured reason identifying which clause failed and what the observed state was at the point of failure.
4. **PRIORITY resolution.** If multiple rules match and produce conflicting verdicts, the rule with the highest PRIORITY value determines the outcome. Higher numeric values indicate higher precedence.
5. **UNKNOWN for unobservable state.** If a predicate term cannot be resolved because no PERCEPTION_MAP in the loaded spec supplies the required observation, emit UNKNOWN with a gap_report identifying the missing term. The evaluator must not substitute a default value for an unobservable term, because doing so would convert a genuine information gap into a false verdict.
6. **RESOURCE_BOUND enforcement for Extended Profile.** If evaluation time or memory consumption exceeds the declared RESOURCE_BOUND before the predicate resolves, halt evaluation and emit UNKNOWN with a gap_report identifying the bound that was exceeded.
7. **Verdict packaging.** Construct the verdict object before returning. All required fields must be populated.

REPORT — the verdict object must have the following structure:

- **status:** one of ALLOW, DENY, or UNKNOWN. No other values are permitted.
- **rule_id:** the canonical RuleID of the GOVERNANCE_RULE that fired. NULL if default-deny.
- **reason:** present if and only if status is DENY. A structured object identifying (a) which clause of the predicate evaluated to FALSE or could not be resolved, and (b) the value of each state variable observed at the point of failure. Reason must be machine-readable, not free-text.
- **gap_report:** present if and only if status is UNKNOWN. A structured object listing either (a) the terms that could not be observed, naming the PERCEPTION_MAP or state variable that was expected to supply them, or (b) the RESOURCE_BOUND that was exceeded, including the declared threshold and the measured value at the point of halt.

The verdict object is the complete output of a single EVALUATE call. The calling system must not rely on any side channel for verdict information.

Determinism guarantee. For the same loaded spec and the same input state, repeated evaluations must produce identical verdicts. A DENY verdict against a given proposed action

on Tuesday must be reproducible against the same input on Friday, given the same loaded spec and the same state object. Any caching, indexing, parallelization, or domain-specific optimization must preserve this determinism guarantee.

Extension governance

The Omega language is designed to grow without breaking existing specifications. Extensions are governed through a structured process:

The Constitutional tier is unamendable. No META_DEFINITION_RULE invocation may target: the 13 primitive declarations, the core EBNF productions of the grammar, the two-profile structure (Strict and Extended) including the static-decidability requirement of Strict, the safety kernel's three-valued verdict shape (ALLOW/DENY/UNKNOWN), or the extension governance rules themselves. A specification that attempts to target these Constitutional elements is not a malformed extension of Omega — it is a different language wearing Omega's syntax.

Five extension layers are defined, each with a different review burden based on reach (how much of the language the extension touches) and reversibility (whether the extension can be deprecated without breaking dependent profiles):

Layer	Examples	Reach	Reversibility	Review Required
Composition patterns	Named reusable composite structures	Low	High	Low
Domain ontologies	Versioned vocabulary for specific application domains	Medium	High	Medium
Resource types	New cost dimensions for RESOURCE_BOUND	Medium	Medium	Medium
Proof protocols	Alternate verification regimes for TRUST_ELEMENT	High	Low	High
Modality extensions	New entries in CONTEXT_RULE MODALITY vocabulary	High	Low	Highest

Every extension is a formal META_DEFINITION_RULE invocation. No parallel mechanism exists. Every extension is scoped to a profile — no extension is implicitly global. A program that does not import the extension must continue to parse, validate, and execute identically to

a program written before the extension existed. Every extension must be backward-compatible at the profile boundary — removing the extension's import must restore the program to a parseable, executable state under the prior vocabulary.

Emergency revocation. A ratified extension may be pulled from the canonical vocabulary through the TRUST_ELEMENT REVOCATION_PROTOCOL applied to the extension's defining META_DEFINITION_RULE. Revocation requires a structured cause and produces a structured remediation path for affected profiles. Revocation is irreversible at the canonical level — a revoked extension may be reproposed only as a new extension with a new identifier and a new rationale record.

Wisdom Harvesting connection

If the Wisdom Harvesting pipeline is deployed, each governance decision — every ALLOW, DENY, or UNKNOWN — is harvested. When a DENY is issued, the system records the pattern: what kind of command was refused, under what conditions, which rule triggered the denial. When an ALLOW is issued, the system records what authorization was present. Over time, the system learns which command patterns are reliably accepted and which are reliably denied. It generates preemptive guidance: "Before sending this type of command, ensure you have the required TRUST_ELEMENT."

When a policy violation narrowly slips through because of ambiguous rule wording, the system harvests the ambiguity and generates a clearer GOVERNANCE_RULE candidate. The Constitution ensures the candidate does not introduce contradictions.

Summary: The Stack as a Whole

Each layer solves a specific failure mode:

Failure Mode	Layer That Prevents It
Infinite regress (systems guess)	Layer 1 — Prime Registry as finite bedrock
Fragmented events across triples	Layer 2 — Synapse as clause-grain atom
LLM hallucination in extracted knowledge	Layer 3 — Deterministic gates between proposal and storage
No discovery mechanism	Layer 4 — Capability manifests
Spoofing, replay, semantic corruption	Layer 5 — HFF five gates

Failure Mode	Layer That Prevents It
Ambiguous intent	Layer 6 — AFP act types + Decider
No structural refusal	Layer 7 — Omega CAN → MAY → DO

Each layer is independently adoptable. You can deploy only the Core Lexicon as a shared dictionary hub. You can add Omega governance for autonomous systems. You can add the Wisdom Harvesting pipeline when your corpus reaches scale. Every layer works alone; every layer composes cleanly.

The Convergent Architecture

The architecture presented here has two ingestion paths, two parallel data models, five lexicon layers, a four-phase alignment pipeline, three depth levels, seven frame types, fifteen semantic roles, two-tier storage, a formally specified wire protocol, a formally specified governance language, a policy-driven Decider with five outcomes, and a deterministic membrane separating reasoning from action.

This complexity is not a design flaw. It is the minimum complexity required to solve the problem. Every component is necessary. None is optional. The architecture was not invented. It was discovered — forced into existence by the constraints of the problem itself.

Remove the Core Lexicon → no shared identifiers → no deterministic verification. Remove the Synapse → TBox/ABox split → identity breaks. Remove the Prime Registry → infinite regress. Remove frames → modality mismatches go undetected. Remove HFF/AFP → every integration is bespoke. Remove Omega → governance must be built externally.

Convergence of independent gauntlets. The 15 thematic roles, the 65 NSM primes, the 8 Synapse link types, and the 13 Omega primitives were not chosen by the author's preference. Each set was discovered through an independent adversarial elimination process — proposing candidates, testing them against structural constraints, keeping only what survived. These four gauntlets operated on different domains (linguistics, cross-linguistic semantics, event representation, governance) and converged independently on stable, irreducible sets. The simplest explanation is that the structure being discovered is real.

Each component exists because a simpler solution failed. The architecture could not have been simpler and still solved the problem.

Capabilities and Compounding

Introduction

The dual-axis architecture described in the Foundations section and the seven-layer stack described in the Architecture sections enable the capabilities described in this section. The value is not in any single layer — it is in what the combined layers make possible that no previous approach can deliver.

This section describes five concrete capabilities that the architecture enables. Each capability maps to one or more pain points from The Problems We Face section. Each shows how SGF layers combine to produce an outcome that is deterministic, verifiable, and governable — and that, if the Wisdom Harvesting pipeline is deployed, improves with use.

Following the five capabilities, this section describes three architectural components that enable compounding: the Semantic CPU (the query interface), Knowledge Packs (the distribution mechanism), and Wisdom Harvesting (the learning pipeline that makes the system improve over time).

5.1 Ontology-to-Ontology Alignment via a Shared Pivot

The pain point it solves

Two organizations have independently built ontologies. Every time they need to exchange data, they must perform a pairwise alignment — mapping terms from Ontology A to Ontology B, negotiating the meaning of shared predicates, testing, validating, documenting. For two ontologies, this is expensive but feasible. For N ontologies, the cost scales as N^2 — every new partner multiplies the integration burden.

This is the root cause of the coalition integration problem (Sections 3.1 and 3.2 in The Problems We Face), the supplier integration problem (Section 3.4), and the data model reconciliation problem faced by every organization that must exchange meaning with multiple partners. It is a direct manifestation of Blind Spot #1 (Ingestion Bottleneck): when every pair of systems requires a separate mapping, the system cannot ingest meaning at the scale that modern operations demand.

How the architecture solves it

Instead of pairwise alignment, each ontology aligns to the Core Lexicon once. A query from Ontology A is resolved to a Core Lexicon canonical ID. The same ID is then resolved to Ontology B. The alignment is a lookup through a shared pivot — not a direct comparison between two independently constructed structures.

Depth is consequence-driven. Not all alignments require the same level of certainty.

Level	Method	Certainty	Use Case
L1	Fingerprint (surface-level matching of canonical IDs and descriptions)	Probabilistic	Browsing, candidate generation, rapid triage
L2	Direct ontology (compare IS_A parents, HAS_PART composition, HAS_ATTRIBUTE one level deep)	Partial structural	Standard procurement, non-critical supply chain
L3	Recursive decomposition (decompose both sides until both hit NSM primes or shared Core Lexicon entries)	Full deterministic	Safety-critical parts, regulatory compliance, legal verification

The same three-level hierarchy applies whether the comparison is between two concepts, two propositions, or two full ontologies. Consequence selects the depth.

Concrete example: pump specification alignment

Consider a scenario the architecture enables. A supplier sends a message: "The PB100 pump meets MIL-STD-810H."

The buyer's system looks up `en.pb100.water_pump_model.noun.custom` — not found in the Core Lexicon. But the message carries a micro-lexicon entry:

```
en.pb100.water_pump_model.noun.custom
    subclass_of
    en.water_pump.machine.noun.core

en.water_pump.machine.noun.core
    subclass_of
    en.pump.machine.noun.core

en.pump.machine.noun.core
    subclass_of
    en.machine.artifact.noun.core
    ← found in shared dictionary
```

The buyer follows the IS_A chain. Each step is a lookup against the Core Lexicon. The chain terminates at `en.machine.artifact.noun.core` — a Core entry that exists in both parties' lexicons. The message resolves.

No prior agreement about "PB100" was needed. No manual ontology mapping occurred. The sender brought its own definition, tethered it to shared ground via binary links, and the receiver followed the links back to the Core.

Full L3 decomposition: a worked example

Consider the attribute "impeller material = stainless steel." L3 decomposition proceeds as follows:

1. Decompose "impeller" down its IS_A chain: impeller IS_A rotating_component IS_A mechanical_part IS_A artifact IS_A THING. Both sides hit THING.
2. Decompose "stainless steel" down its IS_A chain: stainless_steel IS_A steel_alloy IS_A metal IS_A material IS_A SUBSTANCE. Both sides hit SUBSTANCE.
3. Decompose the HAS_PART relation: the supplier's specification lists impeller as HAS_PART pump. The buyer's specification requires impeller as HAS_PART pump. Match.
4. Decompose the HAS_ATTRIBUTE relation: the supplier lists material = stainless_steel (with microgloss `en.stainless_steel.corrosion_resistant_alloy.noun.core`). The buyer requires material = stainless_steel (with microgloss `en.stainless_steel.iron_chromium_alloy.noun.core`). The system checks whether these two microglosses are SAME_AS or NARROWER_THAN/BROADER_THAN. Finding a SAME_AS link in the Core Lexicon, it confirms the match.

The result: a ProofTrace showing each decomposition step, each lookup, each comparison. No guess.

Wisdom Harvesting

If the Wisdom Harvesting pipeline is deployed, each alignment — at every depth level — is harvested. Successful alignments produce reusable IS_A chain patterns. Failed alignments produce GapReports that identify where the chains diverged. Over time, the system accumulates a library of known alignment patterns:

- "Ontologies from this domain generally require L3 for material specifications."
- "This supplier's part numbers consistently follow a different convention than the buyer's."
- "The IS_A chain for this manufacturer's components typically terminates at Machine, not Component."

When a new alignment is requested, the system checks the library first. If a similar alignment pattern exists, it starts at the appropriate depth level rather than defaulting to L1. The system gets faster with every alignment it performs.

5.2 RFP-to-Proposal Compliance Verification

The pain point it solves

A contracting officer receives a request for proposal and a vendor proposal. Determining whether the proposal complies with every requirement is a manual process that takes days or weeks. Requirements are expressed in natural language, and the proposal may satisfy them — or may appear to satisfy them while diverging in critical ways. Modality mismatches ("shall" vs. "will"), constraint mismatches, and frame mismatches (a COMMAND interpreted as an ADVISE) are invisible to current automated systems.

This is the core of the probabilistic part verification problem (Section 3.4), the regulatory compliance traceability problem (Section 3.4), and the machine honesty problem (Section 3.6). It is a direct manifestation of Blind Spot #2 (Consistency Without Truth): current systems cannot determine whether a claim about compliance corresponds to anything real.

How the architecture solves it

GLEAN parses both documents into rooted Synapse Trees — hierarchical structures of requirements (RFP) and offers (proposal), each expressed as Synapses with full epistemic status, modality, and frame. SOAM (Slot-by-slot Orientation Alignment Model) aligns the two trees node by node.

The alignment process examines five dimensions:

1. **Branch matching.** Does the proposal address every section of the RFP? Missing branches are structural absences, not semantic approximations. If the RFP requires a section that the proposal omits entirely, the system flags it as a gap — not a partial match, not a similarity score.

2. **Concept matching.** For each requirement-offer pair, compare IS_A hierarchy, HAS_PART composition, HAS_ATTRIBUTE values, constraints, frames, and modality. The comparison is bidirectional — the offer must satisfy the requirement AND the requirement must accept the offer. A requirement that asks for a "flight-qualified torque driver" is not satisfied by a "hand-held screwdriver rated for atmospheric use," even if both are "tools."

3. **Constraint matching.** Convert units to base SI. Apply the comparison operator with tolerance. A metric passes if the offered value falls within the required range. "3 or more persons" with a minimum of 3 is not satisfied by "2 persons." "Operating temperature -40°C to +85°C" is satisfied by "-50°C to +90°C" (the offered range fully covers the required range). "Weight under 5 kg" is not satisfied by "5.5 kg."

4. **Frame alignment.** COMMAND vs. PROMISE? DUTY vs. ADVISORY? The same structural gap may receive different decisions depending on the frame. A COMMAND that is not

satisfied must be escalated. An ADVISE that is not followed may be acceptable.

5. **Modality alignment.** "Shall" (obligation) vs. "will" (intention) vs. "may" (permission). A requirement stated with "shall" creates a binding obligation. A proposal that matches the content but uses "will" has a modality mismatch — the commitment is weaker than required.

Concrete example: staffing requirement

Consider a scenario the architecture enables. An RFP states: "Kitchen manager shall staff with 3 or more persons during peak hours, all ServSafe certified."

A proposal responds: "Kitchen manager will staff with 2 persons."

SOAM aligns the two:

- **Modality mismatch:** "shall" (obligation) vs. "will" (intention). The proposal uses a weaker commitment than the requirement demands.
- **Constraint mismatch:** ≥3 vs. 2. The offered staffing level does not meet the required minimum.
- **Concept match:** "kitchen manager," "staff," "peak hours" all align. The parties are talking about the same thing.
- **Frame match:** Both are framed as binding commitments (not advisory, not hypothetical).

The Decider applies the Stream of Consciousness policy ("Staffing is essential — this is a safety-critical requirement. Do not accept without binding commitment.") and produces verdict: CLARIFY.

The system sends a structured message to the vendor: "Your proposal states 'will staff with 2 persons.' The RFP requires '3 or more persons' with obligation modality (shall). Do you commit to staffing with 3 or more persons with binding obligation, or do you have a documented justification for the deviation?"

The vendor responds: "We commit to 3 or more persons with binding obligation."

Re-evaluation: modality now matches (obligation to obligation). Constraint now matches (≥3 to ≥3). Verdict: ACCEPT. The system outputs a compliance report with item-by-item ProofTraces.

What the output looks like

For each requirement-offer pair, the system produces:

- **Requirement:** the original text from the RFP
- **Offer:** the matching text from the proposal

- **Alignment status:** FULL_MATCH, PARTIAL_MATCH, MODALITY_MISMATCH, CONSTRAINT_MISMATCH, FRAME_MISMATCH, MISSING, EXTRA
- **GapReport:** if the alignment failed, exactly why — which dimension failed, what the expected value was, what was offered, and what corrective action would resolve the gap
- **Essentiality:** MUST, SHOULD, MAY, INFO
- **Decider verdict:** ACCEPT, REJECT, CONDITIONAL, CLARIFY, ESCALATE

Wisdom Harvesting

If the Wisdom Harvesting pipeline is deployed, every compliance verification is harvested. The system learns:

- Which requirement patterns are most frequently misinterpreted by vendors.
- Which RFP sections are most frequently left non-compliant.
- Which frame alignments require the most CLARIFY rounds.

Over time, the system generates preemptive guidance for RFP authors: "This requirement section has a high historical non-compliance rate. Consider providing explicit definitions for the following terms..." And for proposal authors: "When responding to this RFP section, ensure you match the obligation modality (shall) rather than intention modality (will)."

The verification process gets faster with every document pair processed. The CLARIFY rate drops. The ESCALATE rate drops. The system compounds its compliance expertise.

5.3 The Returns Problem

The pain point it solves

A returned part arrives at a warehouse. It has no packaging. It has no SKU label. It has no documentation. The person at the intake desk must determine what this part is, whether it matches what the customer says they returned, and what to do with it.

Current systems cannot do this. They rely on SKUs, barcodes, or manual lookup. When those are absent, the part sits in a bin, unprocessed, for days or weeks. The returns problem — misidentified parts, restocking errors, lost inventory — is rooted in the inability to identify a part when its surface identifiers are missing.

This is the returns problem from the manufacturing section (Section 3.4 in The Problems We Face) — but the same architecture serves customer support, field service, warranty claims, recall management, counterfeit detection, and secondary market pricing. It is a direct manifestation of Blind Spot #1 (Ingestion Bottleneck): the system cannot ingest the features of the part because it lacks the vocabulary to describe them.

How the architecture solves it

The returns pipeline has three stages, all running on the same SGF stack.

Stage 1: Customer description intake. A customer calls or submits a ticket: "I'm returning a torque driver I bought for my project. It's the blue one with the adjustable clutch."

GLEAN parses the utterance. It extracts the entity: a torque driver. It extracts attributes: color = blue, feature = adjustable clutch. It aligns this description against the product catalog. The alignment may succeed (matched to SKU X) or fail (no match found). If it fails, the GapReport names exactly what is missing — "No product in the catalog has both blue exterior AND adjustable clutch. Products with blue exterior have fixed clutch. Products with adjustable clutch are black." The agent can then probe: "Is the clutch adjustment collar metal or plastic?" Each answer narrows the search.

Stage 2: Receive physical part. The part arrives. It still has no label. The system uses visual features captured at intake — shape, color, connector type, visible markings — and aligns them against the catalog using the same L1-L3 depth hierarchy. L1 surface matching may identify a candidate set. L2 compares HAS_PART and HAS_ATTRIBUTE against known product specifications. L3 decomposes the identifying features until the system can either match to a specific SKU or produce a GapReport.

Stage 3: Route to disposition. Once the part is identified, the system checks its condition against the expected spec. "Surface rust on mounting bracket. Missing rubber foot. Otherwise functional." It produces a disposition recommendation: route to reconditioning, route to recycling, route to customer refund, or flag for manual inspection. The recommendation is governed by the same rules that govern every other decision — the system cannot route a safety-critical component to reconditioning without a certified inspection.

Concrete example: returned torque driver

Consider a scenario the architecture enables. A customer returns a "blue torque driver with adjustable clutch." The catalog shows no such product. The GapReport says: "No match. Blue torque drivers have fixed clutch. Adjustable clutch torque drivers are black."

The agent asks: "Is there a separate adjustment ring near the handle?" The customer: "Yes, it's silver." The system queries the catalog again, this time searching for "torque driver with metal adjustment ring." Three candidates appear. L2 alignment checks IS_A and HAS_ATTRIBUTE. One candidate matches: "Adjustable torque driver, black handle, blue grip sleeve." The grip sleeve is blue. The customer described the grip sleeve as "blue." The system identifies the SKU, checks the customer's order history, finds the purchase, and initiates the return with a ProofTrace.

The customer never guessed. The agent never guessed. The system never guessed.

Wisdom Harvesting

If the Wisdom Harvesting pipeline is deployed, each identification attempt — successful or failed — is harvested. The system learns:

- Which visual features are most discriminative for each product category.
- Which customer description patterns consistently match or fail.
- Which identification failures reveal gaps in the catalog.

Over time, the system gets better at identifying parts with fewer inputs. First-time match rates increase. GapReport depth improves — the system learns to ask the most discriminative question first.

5.4 M2M with Zero Prior Integration — The Stranger Rule in Action

The pain point it solves

A ground station needs to communicate with an allied drone it has never encountered before. The drone was deployed by a different organization, using different communication protocols, different data formats, different classification policies, and a different ontology. Current timelines for this integration: months.

Disaster response, ad-hoc coalition formation, and rapid deployment operations cannot wait months.

This is the first-contact interoperability problem (Sections 3.1 and 3.2 in The Problems We Face). It is a direct manifestation of Blind Spot #1 (Ingestion Bottleneck) and Blind Spot #3 (Open-World Language): the systems cannot ingest each other's meaning because they have never agreed on a shared vocabulary.

How the architecture solves it

Consider a scenario the architecture makes possible. The ground station and the drone have never met. Both have the Core Lexicon. That is the only shared infrastructure.

Step 1: Discovery. The ground station broadcasts a capability manifest. The drone responds with its own manifest. Each learns the other's identity, supported security profiles, supported act types, and lexicon coverage. This happens in seconds, not days.

Step 2: Message construction. The ground station constructs a HIGH_RISK_COMMAND message. The message includes mission parameters (coordinates, time window, sensor configuration), the ontological commitments that define those parameters (coordinate frame = WGS84, time format = UTC, sensor mode = EO), and the governance rules that authorize the

action (COMMAND signed by operator with appropriate authority level, valid within the declared time window, bound to the declared resource constraints).

The message is wrapped in an HFF envelope with content hash, signature, expiry, and nonce. The security profile is CONFIDENTIAL_DIRECT — only the intended recipient can decrypt the payload.

Step 3: Message resolution. The drone receives the message. It passes the five HFF gates: schema valid, hash matches, signature verified, message is fresh, all terms are hydratable against the Core Lexicon plus the sender's micro-lexicon. The drone then evaluates the governance rules carried in the message against its own Omega constitution.

Step 4: Decision. The drone's Omega system checks: does any GOVERNANCE_RULE permit this action? The rule must match the act type (COMMAND), the authority level (operator), the resource constraints (battery, bandwidth, sensor capacity), and the context (time window, operational domain).

If the rule permits the action, the drone executes. It sends back a CONFIRM with a ProofTrace showing which rule permitted the action and what checks were passed.

If the rule does not permit the action — for example, the HIGH_RISK_COMMAND lacks a human TRUST_ELEMENT — the drone returns a GapReport: "HIGH_RISK_COMMAND denied. Required: TRUST_ELEMENT from human operator with appropriate authority level. Missing: authorization frame for autonomous action during degraded network conditions."

The sender cannot compel obedience. The protocol stack prevents it. Receiver Sovereignty is structural.

Wisdom Harvesting

If the Wisdom Harvesting pipeline is deployed, each first-contact interaction produces a partner profile Knowledge Pack. The pack contains:

- The partner's lexicon coverage (which zones and namespaces they participate in).
- Their typical frame patterns (which act types they most commonly use).
- Their governance constraints (which rule patterns they enforce).
- The resolution status of each term encountered (which terms were resolved from Core, which required the partner's micro-lexicon).

Future interactions with the same partner skip the discovery phase and use the cached profile. Future interactions with new partners of a similar type use the accumulated patterns. The 10th new partner integrates faster than the 1st.

5.5 Privacy-Preserving Verification

The pain point it solves

A contractor needs to prove that a component meets a specification without revealing the proprietary design details of the component. An agency needs to verify the claim without requiring access to proprietary data.

Current approaches force a choice: either expose the design data (and lose competitive advantage) or accept the claim on trust (and risk counterfeit or non-compliant parts).

This is the proprietary data exposure problem, and it appears in every domain where a contractor supplies a component to a government or prime integrator. It is a direct manifestation of Blind Spot #2 (Consistency Without Truth): the agency cannot verify the contractor's claim because the data needed to verify the claim is proprietary.

How the architecture solves it

The Exact Profile Contract. The contractor and the agency agree on a profile contract before any data is exchanged. The contract defines:

- Which attributes of the component will be measured (material composition, dimensions, environmental tolerances, manufacturing process certifications).
- The exact measurement methodology for each attribute (which test standard, which instrument, which calibration).
- The LSH fingerprint parameters (which hash function, which bit length, how many hash tables).

The profile contract is part of the governance layer. It is loaded into both systems before verification begins.

The verification process. The contractor runs the tests specified in the profile contract. For each attribute, the measurement produces a value. The value is hashed into the LSH fingerprint. The fingerprint — 86 characters — is sent to the agency.

The agency does not receive the raw measurement. It receives only the fingerprint and a boolean flag for each attribute: PASS or FAIL. The agency can verify that the fingerprint is consistent with the specification — that the contractor's measurement, if correct, would produce a fingerprint that matches the expected range — without ever seeing the actual measurement.

If the agency needs deeper verification — for example, if the fingerprint falls near a boundary — it can request a Zero-Knowledge Proof that the measurement is within the specified range. The ZKP proves the claim without revealing the actual value.

No proprietary design data leaves the air gap.

Concrete example: component verification

Consider a scenario the architecture enables. A contractor produces a titanium alloy bracket for an aircraft. The specification requires: tensile strength ≥ 900 MPa, elongation ≥ 10%, hardness between HRC 30 and HRC 40.

The contractor runs the tests. The results: tensile strength = 945 MPa, elongation = 12%, hardness = HRC 35.

For each result, the contractor:

1. Hashes the value into the LSH fingerprint.
2. Generates a boolean flag: PASS (945 ≥ 900) for tensile strength, PASS (12 ≥ 10) for elongation, PASS (35 is between 30 and 40) for hardness.
3. Sends the 86-character fingerprint and three boolean flags to the agency.

The agency receives: fingerprint = `a1b2c3d4...`, flags = PASS, PASS, PASS. The agency verifies that the fingerprint is consistent with the specification. No raw measurements were exposed. The contractor's proprietary process data is protected.

If the agency later needs to audit the specific tensile strength value — for example, because the part failed in service — it can request a ZKP that the true value was ≥ 900 MPa. The contractor generates the proof without revealing that the value was 945 MPa. The agency learns only what it needs to know: the part met the specification.

Wisdom Harvesting

If the Wisdom Harvesting pipeline is deployed, each verification is harvested. The system learns:

- Which fingerprint parameters are most discriminative for each material and attribute type.
- Which attributes are most frequently challenged (requesting ZKP verification) and which are accepted on fingerprint alone.
- Which contractors have a consistent track record of PASS flags and which require more frequent auditing.

Over time, the system adjusts verification depth based on historical patterns. A contractor with many consecutive PASS flags for similar components may be accepted at fingerprint-only depth. A new contractor may require ZKP verification for early batches. The system balances verification rigor against verification cost — and improves with every interaction.

5.6 The Semantic CPU

What it is

The Semantic CPU is the query and reasoning interface to the Synapse graph. Given a query expressed as a Synapse pattern, it walks the graph, follows roles and links, applies epistemic status filters, checks Omega rules when governance is present, and returns results with full provenance.

It does not predict tokens. It resolves canonical IDs, follows IS_A chains, traverses role bindings, and returns what it finds. If the structure does not contain the answer, it returns a GapReport explaining exactly what is missing.

Comparison to LLMs

Dimension	LLM	Semantic CPU
Source citation	None, or hallucinated	Specific, with document and page
Verifiability	Impossible	Full ProofTrace
Currentness	Training cutoff	Knowledge Pack version
Update cost	Millions (retraining)	A file download
User trust	"It sounds right"	"I can check the source"

How it works

1. A query arrives expressed as a Synapse pattern — a VerbHub with some roles filled and some left open.
2. The Semantic CPU walks the Synapse graph, matching the pattern against stored Synapses.
3. It applies epistemic status filters — results below a configurable threshold (e.g., PROVISIONAL) are excluded by default.
4. It checks Omega rules if governance is present — some queries may be denied because the question exceeds the asker's authorization.
5. It returns the matching results with full provenance: source document, section, sentence, offset.
6. If no match is found, it returns a GapReport: "No Synapse matching this pattern exists. The closest patterns were [X] and [Y], which differ on roles [Z]."

The cold start problem

The first Semantic CPU will have no Knowledge Packs to load. The solution is to start small and specific:

1. **Bootstrap the Core Lexicon** from open sources — automated, takes hours.
2. **Compile the first domain pack** — one well-defined, authoritative domain.
3. **Build the query engine** — just the query layer. Load the domain pack. Show deterministic, provenance-bearing answers.
4. **Release both as open source.**
5. **The ecosystem follows.**

One domain. One pack. One query engine. Prove the concept. Then scale.

5.7 Knowledge Packs

What they are

A Knowledge Pack is a signed, versioned, queryable compilation of knowledge — the distribution mechanism for SGF-compiled meaning. A law firm does not need to train a model on the Louisiana Civil Code. They buy a Knowledge Pack — a compilation of the Code, compiled into Synapses by domain experts.

The pack includes:

- Lexicon entries for every defined term
- Synapses for every rule and exception
- IS_A chains grounding every term back to the shared Core Lexicon
- Provenance for every claim

The pack is loaded into a local graph store. Questions are answered with ProofTraces: "Section 47:101 of the Louisiana Civil Code states that..." No hallucination. No opaque reasoning.

Knowledge Packs are independent, versioned, signed by their issuer, and loadable into any SGF-compatible graph store. Any domain expert can produce a pack. Any user can choose which packs to trust.

Knowledge Packs vs. Pluggable Brains

The architecture supports two distinct kinds of pluggable modules, both stored as signed, versioned Packs:

Knowledge Packs contain grounded facts about a domain. A Knowledge Pack for Louisiana law contains Synapses representing each statute, each court decision, each regulatory requirement — stored as a queryable Synapse graph with full provenance chains. A Knowledge Pack for aircraft maintenance contains the parts catalog, the maintenance procedures, the FAA regulations. Knowledge Packs answer: "What is true in this domain?"

Pluggable Brains contain behavioral wisdom — motivation rules that govern how the AI thinks, reasons, and behaves. A Pluggable Brain for safety-critical operations contains the rules that ensure autonomous systems never comply with an unsafe command, that uncertainty is surfaced before confidence, and that second-order effects are traced before optimization. A Pluggable Brain for creative writing contains rules that govern narrative structure, voice, and revision strategy. Pluggable Brains answer: "How should I think in this domain?"

Both use the same retrieval infrastructure. A Knowledge Pack is loaded, and its facts are available to the Semantic CPU. A Pluggable Brain is loaded, and its motivation rules are injected into the governance layer. Both can be combined — a drone loading a tactical Knowledge Pack (target classifications, no-fly zones) alongside a safety-critical Pluggable Brain (prioritizing human verification of lethal action, refusing commands that lack required authorization).

A starter Pluggable Brain — "The Operating System of a Mind" — contains approximately 600 rules organized into 9 layers, each correcting a specific default LLM failure mode. It can be loaded today. Any system that loads it will immediately exhibit structural refusal, epistemic honesty, calibrated uncertainty, creative exploration, and long-term coherence.

The closed vocabulary at the core of SGF — 65 NSM primes, 15 semantic roles, 7 binary relations — serves as the shared substrate for both. A Knowledge Pack's facts and a Pluggable Brain's rules are both expressed as Synapses. The difference is not in the representation. It is in the function: facts inform. Rules govern.

5.8 Wisdom Harvesting: The Compounding Dimension

Why this layer exists

The seven-layer stack solves the static problem: how to represent, move, verify, and govern meaning across boundaries. A system with those seven layers can exchange meaning deterministically, refuse unauthorized commands, and produce ProofTraces for every decision.

But it cannot **learn**.

A system that cannot learn repeats its mistakes. Every failed alignment, every refused command, every CLARIFY round, every compliance gap — each is a lesson that, if captured, could prevent future failures. Without a learning mechanism, the system is limited to the knowledge that was loaded into it at deployment. It does not compound.

The AI OS layer — Wisdom Harvesting — solves this. It sits above the seven-layer stack, monitoring every interaction, extracting cross-domain principles, storing them in a retrievable

corpus, and surfacing them when relevant situations arise. The kernel is stable. It does not modify itself. It grows by accumulating wisdom, not by changing its own code.

The architecture of the AI OS

The AI OS has four components that work together to harvest, store, retrieve, and apply wisdom.

The Stable Kernel. The kernel is the smallest, most auditable part of the system. It contains:

- The **Prime Registry** (65 NSM primes — Layer 1).
- The **Core Lexicon** (shared dictionary — Layer 2).
- The **Omega Constitution** (13 Constitutional primitives — Layer 7).
- The **Event Horizon** gate (separating probabilistic reasoning from deterministic action).

The kernel never self-modifies. It cannot be changed by any harvested rule. It is the fixed point that guarantees the system remains predictable, auditable, and certifiable across its entire operational lifetime.

The Wisdom Vault. The Wisdom Vault stores harvested knowledge. It is a corpus that grows without bound. No manual curation is required at ingest time. Every harvested lesson is stored as-is, with a confidence score starting at 0.5. The vault has three tiers:

Tier	Contents	Confidence Threshold	TTL
Active	Rules that are frequently retrieved and produce good outcomes	≥ 0.7	Indefinite
Candidate	Rules that have been retrieved at least once but not yet proven	0.5 – 0.7	90 days
Archived	Rules that were never retrieved or were superseded	< 0.5	Indefinite (audit trail)

The vault is governed by the same Omega rules that govern every other part of the system. A rule that would violate the Constitution — for example, a harvested rule that suggests weakening a safety constraint — is rejected at ingest time.

Each entry in the Wisdom Vault has the following structure:

Field	Description
rule_text	The display string — what gets surfaced to the agent or user
embedding_payload	Concrete instantiation block — the text that anchors the rule in vector space for retrieval
applies_when	Natural-language description of the situations this rule applies to
domain	Which domain(s) the rule belongs to
confidence	How reliable the rule has proven in practice (0.0 to 1.0)
source	Where the rule came from
retrieval_count	How many times the rule has been used
governance_status	PRE_HARVEST, VETTED, SUPERSEDED, ARCHIVED

The embedding_payload field is critical for abstract rules. A rule like "release finite shared resources after use" has no natural location in vector space because it is abstract. The embedding_payload contains three to five concrete instantiations across diverse domains:

- "Closing a database connection after a query prevents connection pool exhaustion."
- "Releasing a file handle after reading prevents file descriptor exhaustion."
- "Unlocking a mutex after a critical section prevents deadlock."

These concrete examples embed well. When a new situation involves any of these concrete activities, the situation's embedding finds the instantiation block, and through it the abstract rule. The rule is retrieved not because anything in the situation resembles the rule's abstract words, but because the situation resembles one of the rule's concrete instantiations.

The Harvesting Pipeline. The harvesting pipeline monitors the system's ongoing interaction stream — conversational transcripts, pipeline execution logs, quality gate reports, user corrections, debugging sessions, retrospective analyses. It selects segments that contain teachable moments and sends them to an LLM with a structured prompt.

The prompt asks for two forms of each lesson:

1. **A domain-specific rule** — written in concrete language about the specific situation.
2. **A cross-domain rule** — abstracting the principle to a domain-agnostic form.

The LLM returns zero or more lesson pairs. The cross-domain rule is the treasure. It can apply in a thousand different situations across domains that have nothing to do with the original

context.

Each harvested lesson is anchored — the cross-domain rule is sent to another LLM call that generates three to five concrete instantiations across diverse domains. These become the embedding_payload. The rule is then stored in the Wisdom Vault.

The harvesting pipeline runs as a background process during idle time. It is interruptible — if a user task arrives, the harvest yields immediately.

The Retrieval Engine — The Closed-Vocabulary Bridge. The Wisdom Vault can grow to millions of entries. Retrieving the right rules for a given situation cannot depend on brute-force similarity search. The Closed-Vocabulary Bridge solves this by inserting a finite, named intermediate vocabulary between the unbounded wisdom corpus and the unbounded incoming situations.

- Three tables, one LLM call per situation:*

1. `problem_classes` — a closed vocabulary of classes derived by clustering the wisdom corpus. Each class has a name, a one-paragraph description, and three to five concrete example situations. The vocabulary is finite — typically 100-200 classes at the top level.
2. `wisdom_class_bindings` — a bridge table mapping each wisdom entry to one or more classes with a strength weight (0.0 to 1.0).
3. **The per-situation classifier** — when a new situation arrives, one LLM call selects one to five classes from the closed list. A SQL join returns the wisdom entries bound to those classes. The result is a manageable set of applicable rules — typically 5 to 30 items.

The closed-set assumption is what makes this reliable. The LLM is doing recognition (picking from a list shown in its context), not generation (producing a query against an unbounded corpus).

The Graduation Pipeline. A wisdom entry that is repeatedly retrieved across multiple independent situations and consistently produces good outcomes is a candidate for graduation — promotion from the Wisdom Vault to the permanent Omega rule set.

The graduation criteria:

Criterion	Threshold	Why
Age	≥ 30 days in the Active tier	Prevents premature graduation
Retrieval breadth	Used in ≥ 5 distinct tasks	Proves generalization

Criterion	Threshold	Why
Confidence	≥ 0.8	Proves reliability
No unresolved conflicts	Passes RULE_CONFLICT_DETECTION	Proves consistency
Human review	A human explicitly approves	Final safety gate

When these criteria are met, the entry is proposed for graduation. A human reviews the proposal and approves, rejects, or modifies it. Graduated rules enter the Omega governance layer with `status='vetted'` and `source='wisdom_graduation'`.

This is the path from a single observed failure — "the database connection was left open and the pool exhausted" — through the cross-domain principle "release finite shared resources after use," through repeated successful retrieval across database, file handle, mutex, and API rate-limit contexts, to a permanent rule in the system's behavioral constitution.

The Retirement Pipeline. A wisdom entry that is never retrieved is a candidate for retirement. The system periodically audits the vault for entries with zero or near-zero retrieval counts over sustained periods. Retired entries are archived, not deleted. The full history is preserved for audit and potential reactivation.

Concrete example: a harvested rule across the lifecycle

Step 1 — The incident. A system operator forgets to close a database connection. The connection pool exhausts. The pipeline hangs. The operator corrects the issue manually.

Step 2 — The harvest. The harvesting pipeline selects this segment and sends it to the LLM:

- **Domain-specific rule:** "After querying the database, always close the connection handle to prevent connection pool exhaustion."
- **Cross-domain rule:** "After completing an operation that acquired a finite shared resource, release the resource to prevent exhaustion for other consumers."

Step 3 — Anchoring. The LLM generates five concrete instantiations:

- "Closing a database connection after a query prevents connection pool exhaustion."
- "Releasing a file handle after reading prevents file descriptor exhaustion."
- "Unlocking a mutex after a critical section prevents deadlock."
- "Returning borrowed memory to the allocator prevents heap fragmentation."
- "Closing an API session after the request completes prevents rate-limit saturation."

Step 4 — Storage. The rule enters the Wisdom Vault with confidence 0.5. The vocabulary bootstrap has already run. This rule's applies_when description is clustered with other rules about resource management. The cluster is named `resource_lifecycle_management`.

Step 5 — First retrieval. A week later, a different operator is debugging a file descriptor exhaustion issue. The system classifies the situation as `resource_lifecycle_management`. The SQL join returns the harvested rule. The operator applies the principle. The issue is resolved. The rule's confidence rises to 0.55.

Step 6 — Repeated retrieval. Over six months, the rule is retrieved in 12 additional situations: mutex deadlocks, memory fragmentation, API rate limits, semaphore contention, connection pool exhaustion, cache slot exhaustion. Its confidence rises to 0.85.

Step 7 — Graduation. The system detects that the rule meets all graduation criteria. A human reviews and approves. The rule enters the Omega rule set as a GOVERNANCE_RULE. It is no longer a suggestion — it is a constraint.

The ecosystem effect

Wisdom Harvesting works within a single system. But its full power emerges when Knowledge Packs are shared across organizational boundaries.

Anonymized Wisdom Packs enable cross-organizational learning without exposing proprietary or classified data. The pack contains only the cross-domain rule, its concrete instantiations (abstracted from the specific contexts), and its retrieval history. It does not contain the original transcripts, the specific system names, or any identifying information.

An autonomous system learns from a manufacturer's counterfeit detection failure — not because the manufacturer shared its supply chain data, but because it published an anonymized Wisdom Pack containing the cross-domain principle: "When a supplier's provenance chain has been incomplete for two or more consecutive batches, escalate verification depth to L3." The drone applies this principle to its own supply chain verification.

The ecosystem compounds its collective wisdom. Each anonymized pack benefits every organization that loads it. The rate of learning accelerates with every participant.

Summary: What the Architecture Makes Possible

Capability	Pain Points Addressed	Key Layers Used	Gets Better With Use?
Ontology-to-ontology alignment via shared pivot	N² integration cost, coalition interoperability	1, 2	Yes, if Wisdom Harvesting deployed
RFP-to-proposal compliance verification	Probabilistic verification, regulatory traceability	2, 3, 6, 7	Yes, if Wisdom Harvesting deployed
The returns problem	Returns problem, part identification	2, 3	Yes, if Wisdom Harvesting deployed
M2M with zero prior integration	First-contact interoperability, rapid coalition formation	4, 5, 6, 7	Yes, if Wisdom Harvesting deployed
Privacy-preserving verification	Proprietary data exposure, supply chain trust	1, 2, 7	Yes, if Wisdom Harvesting deployed

Every capability is available today. Every capability improves with use if the Wisdom Harvesting pipeline is deployed. No capability requires prior agreement between the parties beyond the shared Core Lexicon.

Adoption, Credits, and Close

6.1 Incremental Adoption — No Rip-and-Replace

SGF is designed to be adopted one layer at a time. You do not need to deploy the full stack to get value. Every layer works independently. Every layer composes cleanly with the layers above and below it.

The adoption path has four phases. Each phase delivers independent value. You can stop at any phase. You can start at any phase. You can skip phases.

Phase 1: The Core Lexicon as a Shared Dictionary Hub

What you deploy:

- The Prime Registry (65 NSM primes).
- The Core Lexicon (bootstrapped from WordNet, Wiktionary, and Wikipedia — or your own dictionary).
- The five-zone lexicon architecture (Core, Inferred, Custom, Instance, Ghost).
- GLEAN, the prose-to-graph compiler, if you have unstructured documents to ingest.

What it costs:

- Automated bootstrap: a few hours of compute to assemble the Core Lexicon from open sources.
- If you already have a dictionary, alignment to the Core: one-time mapping effort proportional to your dictionary's size.

What it delivers:

- A shared reference point that every system in your organization can tether to.
- The ability to resolve previously unknown terms via IS_A chains instead of guessing.
- Deterministic gap reports when a term cannot be resolved — instead of a silent hallucination.
- Ghost Protocol: the system says "I know this exists, but I do not yet know what it is" instead of fabricating.

Who should start here:

- Any organization that needs to integrate multiple data sources or knowledge graphs.
- Any organization that exchanges specifications, requirements, or product data with external partners.
- Any organization that is tired of manual ontology mapping.

Adoption is incremental. You do not need to migrate existing systems. New data flows through GLEAN. Legacy data is preserved. The Core Lexicon grows as you use it. The mapping registry — legacy URIs to canonical IDs — is built incrementally as each legacy system is accessed.

Independent value proposition: Even without any other SGF layer, the Core Lexicon eliminates the most expensive failure mode in semantic integration: the inability to resolve unknown terms deterministically.

Phase 2: Governance — Omega for Autonomous Systems

What you deploy:

- The Omega governance language (13 Constitutional primitives).
- The CAN → MAY → DO gate.
- The safety kernel (Strict profile for safety-critical rules, Extended profile for complex policy).
- The Event Horizon (separating probabilistic reasoning from deterministic action).
- Receiver Sovereignty gates on all incoming HFF/AFP messages.

What it costs:

- Authoring the Constitutional rules for your system (one-time effort, proportional to the complexity of your safety constraints).
- Compiling and testing the rules before deployment.

What it delivers:

- A machine that can refuse a command that violates its constitution — even if the command is properly signed, fresh, and authorized by a human operator.
- Every action is governed by a compiled rule, not a probabilistic policy.
- Every refusal produces a GapReport naming exactly what authorization is missing.
- The system is auditable — every governance decision has a warrant.

Who should start here:

- Organizations deploying autonomous systems.
- Missions requiring validation without human-in-the-loop.
- Industrial robotics operations where safety rules must be verifiable.
- Any organization deploying autonomous systems in safety-critical or security-critical environments.

Adoption is incremental. You can deploy Omega to govern a single system first. You can extend governance to M2M communication via HFF/AFP in Phase 3. You do not need the Core Lexicon if your system uses a fixed vocabulary — but governance is more powerful when it can reason about terms it has not seen before.

Independent value proposition: Omega is a governance language designed specifically for autonomous systems. It is non-Turing-complete, compiled at load time, and constitutionally constrained — a combination that gives you structural refusal with an auditable warrant.

Phase 3: Communication — HFF/AFP for M2M Integration

What you deploy:

- The HFF wire protocol (content hash, signature, expiry, nonce, micro-lexicon hydration).
- The AFP act layer (13 illocution types, conversation state, authority validation).
- Security profiles (PUBLIC_SIGNED_BROADCAST, CONFIDENTIAL_DIRECT, CONFIDENTIAL_GROUP, HIGH_RISK_COMMAND).
- The Stranger Rule for first-contact communication.

What it costs:

- Implementing the HFF/AFP protocol in your systems. The RFCs are published. Reference implementations are available.
- Bootstrapping trust anchors for participating systems and signing authorities.

What it delivers:

- Two machines that have never met can exchange meaning on first contact — including mission parameters, ontological commitments, and governance constraints.
- Every message is integrity-checked, authenticated, freshness-validated, and hydration-verified before it is admitted.
- A HIGH_RISK_COMMAND cannot be spoofed, replayed, or injected.
- Integration time for new partners drops from months to seconds.

Who should start here:

- Organizations that need to coordinate across coalition partners.
- Organizations deploying multi-national autonomous systems.
- Supply chain integrations across multiple tiers and multiple organizations.
- Any organization that needs to exchange meaning with partners it has not pre-integrated.

Adoption is incremental. Phase 1 (Core Lexicon) is not strictly required for HFF/AFP — but the Stranger Rule is far more powerful when both parties share the Core. If you deploy HFF/AFP without the Core Lexicon, you must rely on the sender's micro-lexicon for all non-shared terms, which works but increases message size and processing cost.

Independent value proposition: HFF/AFP provides semantic integrity guarantees — not just transport integrity, but meaning integrity. The receiver knows that the meaning that arrived is the meaning that was sent, because every term is hydratable against a shared lexicon or the sender's explicit definitions.

Phase 4: Wisdom Harvesting — The Compounding System

What you deploy:

- The harvesting pipeline (monitoring the interaction stream, selecting teachable moments, extracting domain-specific and cross-domain rules).
- The anchoring pipeline (generating concrete instantiations for cross-domain rules).
- The Wisdom Vault (storing harvested rules with confidence, provenance, and retrieval history).
- The Closed-Vocabulary Bridge (classifying situations against a finite vocabulary and returning applicable rules via SQL join).
- The graduation pipeline (promoting proven rules to the Omega rule set).
- The retirement pipeline (archiving rules that no longer serve).

What it costs:

- Bootstrap: running the vocabulary derivation over your accumulated wisdom corpus (a few hours of compute for a corpus of 10,000+ entries).
- Per-harvest: one LLM call per transcript segment using the fast model tier.
- Per-rule anchoring: one LLM call per cross-domain rule, paid once.
- Per-retrieval: one LLM call per situation (the classifier) plus one SQL join.

What it delivers:

- The system learns from every interaction, every failure, every success — and compounds its expertise over time.
- Lessons from one domain are automatically surfaced in other domains where the same structural pattern applies.
- Knowledge survives personnel rotation — Wisdom Packs preserve the lessons of departing experts.
- Knowledge survives hardware failure — Lifeboat Packs load the wisdom of a failed system into its successor.
- Cross-organizational learning through anonymized Wisdom Packs.

Who should start here:

- Organizations that have already deployed Phase 1-3 and have an accumulated corpus of interactions.
- Organizations that face rapid personnel rotation and need institutional memory that persists.
- Organizations that need to propagate lessons across organizational or geographical boundaries.

Adoption is incremental. Wisdom Harvesting requires at least the Core Lexicon (Phase 1) for anchoring and retrieval. It is far more powerful with Omega governance (Phase 2) because graduated rules become enforceable constraints. It is most powerful with HFF/AFP (Phase 3) because Wisdom Packs can be distributed as signed, authentic Knowledge Packs. But you

can start harvesting without governance — rules stored in the Wisdom Vault are available as guidance even before they graduate to enforcement.

Independent value proposition: Wisdom Harvesting compounds expertise without modifying the system's kernel or constitution. The kernel stays stable and certifiable. The harvest grows above it.

Phase Transition Summary

Phase	What You Deploy	Independent Value	Prerequisites
1	Core Lexicon + GLEAN	Deterministic term resolution. No more guessing.	None
2	Omega governance	Structural refusal. Auditable warrants. CAN → MAY → DO.	None
3	HFF/AFP + Stranger Rule	Zero-prior-integration M2M. Semantic integrity guarantees.	Core Lexicon (recommended, not required)
4	Wisdom Harvesting pipeline	Compounding expertise. Knowledge that survives personnel and hardware change.	Core Lexicon required. Omega recommended.

Each phase is independently valuable. Each phase composes with the phases before it. You can start at Phase 1 and stop. You can start at Phase 3 and add Phase 2 later. You can start at Phase 4 and work backward. The architecture supports any adoption path.

6.2 Implementation Status

The architecture is real. The code is open source. The standards are published.

What exists today

Component	Status	License	Repository
Core Lexicon bootstrap	Working	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest

Component	Status	License	Repository
scripts			
Prime Registry	Published as RFC	Open standard	RFC format in manifest
Synapse data model	Specified in full	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest
GLEAN prose-to-graph pipeline	Working	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest
HFF protocol specification	Published as RFC	Open standard	RFC format in manifest
AFP act type specification	Published as RFC	Open standard	RFC format in manifest
Omega governance language	Specified in full	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest
SOAM alignment engine	Specified in full	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest
Knowledge Pack format	Specified in full	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest
Wisdom Harvesting pipeline	Specified in full	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest
Closed-Vocabulary Bridge	Specified in full	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest
Starter Pluggable Brain (RFC-XXX)	Published	Apache 2.0	github.com/SymbolGroundingFramework/SGF-manifest

What is under active development

Component	Status	Expected
Semantic CPU reference implementation	In progress	Q4 2026
HFF/AFP reference implementation	In progress	Q4 2026
Omega compiler	In progress	Q1 2027
Wisdom Vault with Closed-Vocabulary Bridge	In progress	Q1 2027
BFO formal alignment review	Planned	Q2 2027

The reference implementation is Apache 2.0 licensed. The patent pledge is perpetual. The protocol specifications are published as open standards in RFC form. There is no licensing fee, no subscription, no vendor lock-in. The architecture is the standard, not the implementation.

6.3 The Synapedia: A Convenience Dictionary

SGF provides an architecture for shared dictionaries. It does not mandate any particular dictionary. However, building one from scratch is expensive. SGF provides a convenience dictionary called the **Synapedia**, assembled automatically from three open sources:

- **WordNet** — core vocabulary with sense distinctions (~150,000 English words)
- **Wiktionary** — long-tail coverage across hundreds of languages
- **Wikipedia** — world knowledge: named entities, scientific concepts, historical events

The scripts are published. The process is transparent. Any organization can assemble the Synapedia in hours, producing a working dictionary with canonical IDs, sense distinctions, IS_A chains tracing every concept to one of 65 semantic primes, and precomputed embeddings.

Organizations that already have their own dictionary are free to use it instead. The dictionary is a parameter, not a constraint.

6.4 Alignment with Formal Ontology Standards: BFO, CCO, and NSOF

SGF is designed to be compatible with realism-based ontology frameworks. The architecture maps to the NSOF's three-tier structure:

NSOF Tier	Standard	SGF Role
Top-Level (BFO)	Basic Formal Ontology (ISO 21838-2)	Grammar designed for BFO alignment: 5 dependence relations, continuant/occurrent distinction
Mid-Level (CCO)	Common Core Ontologies	Synapedia as shared pivot lexicon; SOAM aligns local terms to common core
Domain-Level	Specialized extensions	GLEAN populates domain instances; Custom and Inferred zones hold tethered terms

BFO Mapping (Work in Progress). The following table shows how SGF concepts map to BFO categories. This is a living artifact, updated as alignment deepens.

SGF Concept	BFO Category	Status
Synapse (event)	Occurrent (process)	Mapped
Entity (Core Lexicon entry)	Continuant (independent)	Mapped
HAS_ATTRIBUTE (material, color, etc.)	Quality	Mapped
VerbHub + modality	Disposition	Mapped
IS_A	Subclass of (universal)	Mapped
HAS_PART	Has part	Mapped
Epistemic status hierarchy	Not in BFO (metalanguage layer)	Extension
Frame system (Act, Normative, etc.)	Not in BFO (interpretation layer)	Extension

In February 2024, the Department of War and Intelligence Community formally adopted BFO and CCO as baseline standards for all formal ontology development. SGF provides the operational layer that makes those standards work at scale. It does not compete with CCO. It makes CCO operational.

Important note on BFO alignment status. This mapping has not been submitted for formal review by the BFO community or NCOR, and I do not claim that SGF is currently BFO-compliant. Alignment with BFO is a foundational goal of this project and a work in progress. The BFO mapping in this paper is SGF's own work and has not been reviewed or endorsed by

the BFO community, NCOR, or any of its members. I hope that the community will read this paper and tell me where the mapping falls short.

6.5 Honest Claims

Overclaim	Technical Reality
"Eliminates schema drift"	Localizes drift; makes it auditable via the Usage Ledger.
"Eliminates integration debt"	Reduces N^2 to N-to-Core where Core coverage exists.
"Fully private"	Selective disclosure via SurfaceArea. Core is public; private lexicons are private.
"BFO-compliant out of the box"	Work in progress. Mapping table above shows current status.
"Never hallucinates"	Returns GapReport when it cannot answer. That is structured honesty.
"Works for all prose"	Not all prose is suitable. Joyce's *Ulysses* resists paraphrase. The system lowers confidence and emits more GapReports.
"Eliminates the need for LLMs"	LLMs serve a bounded, subordinate role in parsing only. They do not participate in reasoning, alignment, or governance.
"Solves the symbol grounding problem"	Provides an engineering solution that works at scale. Philosophical questions about whether this "solves" the problem will persist.

6.6 Foundational Principles

Principle	Meaning
15 closed roles, stable grammar, open vocabulary	The roles are closed. The grammar is stable. The vocabulary is infinite.

Principle	Meaning
Clause-grain atom	Triples are too small; paragraphs are too large. The clause is the unit.
Three-axis theorem	Identity requires ontology (Y) + properties (P) + events (X). No two are sufficient. The four Roosevelts prove this.
Finite Bedrock Principle	Every unbounded domain needs a finite floor. 65 primes terminate recursion.
Honest uncertainty	GapReport or Clarification instead of fabrication. Silence is better than confident nonsense.
Receiver sovereignty	The receiver decides. The sender cannot compel admission.
No destructive merge	SAME_AS is a suggestion. Bridge Map is the truth. Nodes are never destroyed.
Metonymic patterns first, then VerbHub	The patterns determine the sense; the VerbHub determines the role. The grammar decides, not the embedding.
CAN → MAY → DO is structural	Enforced at load time, not runtime.
Conservation Law	Every meaning crossing requires pivot, bridge, policy, and proof.
Convergence of independent gauntlets	The 15 roles, 65 primes, 8 link types, and 13 Omega primitives were each discovered through independent adversarial elimination processes that converged on stable, irreducible sets. The simplest explanation is that the structure being discovered is real.

6.7 Influences and Credits

Every idea in this architecture stands on the shoulders of others. I name those debts explicitly here. A note on how to read this section: it credits work that shaped my thinking. It does not claim endorsement by any of the people or institutions named. Any errors in how I have applied their ideas are mine alone.

The Natural Semantic Metalanguage (NSM) research community. The 65 semantic primes that form SGF's Prime Registry were not invented by this project. They are the result of more than fifty years of cross-linguistic research conducted by Anna Wierzbicka, Cliff Goddard, and the international NSM community. Wierzbicka's foundational work — beginning with her 1972 study, extending through *Semantics: Primes and Universals* and *What Did Jesus Mean?* — demonstrated that all human languages share a small set of semantically irreducible concepts. Goddard's subsequent research across more than sixteen language groups validated and refined the set. Reading this literature convinced me that a finite grounding floor for machine meaning was not merely desirable but empirically defensible. The Finite Bedrock Principle is my attempt to apply their discovery to machine semantics. The discovery is theirs; the application is mine, and they bear no responsibility for how I have used it.

Jeffrey Gruber, Charles Fillmore, and Ray Jackendoff. The 15 thematic roles that form SGF's closed grammar were introduced into theoretical linguistics in the 1960s by Gruber and Fillmore, and further developed by Jackendoff in his 1972 work on semantic interpretation. Fillmore's Frame Semantics and its computational implementation, FrameNet, showed me how role-bound event structures could be built at scale. SGF closes the role set at 15, but the concept of thematic relations is entirely theirs.

Barry Smith, John Beverley, and the National Center for Ontological Research at the University at Buffalo. Reading the writings of Smith, Beverley, and their colleagues at NCOR — on realism-based ontology, on the Basic Formal Ontology, and on what it means for an ontology to be founded in how the world actually is — made me aware of the importance of a rigorous top-level ontological framework. BFO, standardized as ISO 21838-2, has shaped the direction of SGF's grammar and continues to shape it. The more I studied BFO, the more I recognized that decisions I had made for engineering reasons — the separation of continuants and occurrents into separate axes, the Instance Minting Rule, the handling of roles as dependent entities — had direct counterparts in BFO's formal axiomatic system. I want to be precise about where this stands: I have not submitted SGF for formal review by the BFO community or NCOR, and I do not claim that SGF is currently BFO-compliant. Alignment with BFO is a foundational goal of this project and a work in progress. I hope that the community will read this paper and tell me where the mapping falls short.

The Common Core Ontologies (CCO) project. The CCO team's mid-level ontologies for defense, intelligence, and cybersecurity provide a shared vocabulary that SGF's Synapedia is designed to complement. The 2024 adoption of BFO and CCO as baseline standards by the Department of War and the Intelligence Community created the institutional context in which SGF's operational layer could be useful.

The National Security Ontology Foundry (NSOF). The Foundry's three-tier structure — BFO at the top, CCO at the middle, domain ontologies at the bottom — is a deployment model that informs how SGF positions itself as an operational layer rather than a competing standard.

Bruno Baumgartner. Our conversations about deterministic knowledge infrastructure at DigiEmu made me aware of two gaps in SGF: state integrity verification (the ability to prove that a Synapse graph has not been tampered with after storage) and privacy-preserving disclosure tiers. His work on SKC, VSC, and the DigiEmu Core architecture clarified the boundary between semantic content and state integrity. SGF is stronger because of those conversations.

Dr. Nicholas Figay. His work on semantic cartography for industrial interoperability — "inhabiting Babel," as he frames it — influenced SGF's approach to cross-boundary meaning transfer. His observation that SGF's grammar lacked a CONTAINS relation was correct and has been addressed.

The open source communities behind WordNet, Wiktionary, and Wikipedia. The Synapedia convenience dictionary is built on their labor. The architecture of the Synapedia is SGF's; the knowledge is theirs.

The Apache Software Foundation and the open source legal tradition. SGF's perpetual patent pledge and Apache 2.0 licensing follow the example of Tim Berners-Lee, Vint Cerf, Bob Kahn, and Linus Torvalds — people who chose adoption over ownership.

Wesley Hohfeld. The eight fundamental legal relations — rights, duties, privileges, no-rights, powers, liabilities, immunities, disabilities — provide the conceptual framework that Omega's governance model extends beyond the first four.

Finally, I thank everyone who read early drafts, pointed out gaps, and asked hard questions. If I have omitted someone who influenced this work, the omission is mine. Please tell me, and I will correct it.

6.8 How to Get Involved

This is an open invitation to collaborate. The code and RFCs are on the GitHub repository. The standards are published and free.

Try the code. Clone the Synapedia bootstrap. Run it against your own documents. Tell us what breaks. The repository includes test data, documentation, and a community issue tracker.

Map a domain. Pick a section of your favorite regulation, specification, or operational manual. Compile it into Synapses using GLEAN. Test the query engine. Submit your mapping as a Knowledge Pack. If you are a domain expert, your Knowledge Pack is the most valuable contribution you can make.

Write a Pluggable Brain. A Pluggable Brain is a collection of motivation rules — behavioral wisdom that governs how an AI thinks, reasons, and behaves. Unlike a Knowledge Pack (which contains grounded facts about a domain), a Pluggable Brain contains the principles, heuristics, and correctives that make an AI trustworthy, reliable, and honest.

A starter Pluggable Brain — "The Operating System of a Mind" — is published as RFC-XXX. It contains approximately 600 rules organized into 9 layers, each correcting a specific default LLM failure mode. You can load it today and your AI will immediately exhibit structural refusal, epistemic honesty, and creative exploration.

But this is just one brain. The architecture supports many. You could author:

- A **safety-critical operations** brain for autonomous systems that must never comply with an unsafe command.
- A **negotiation** brain for procurement agents that must recognize bad faith and protect leverage.
- A **medical triage** brain for diagnostic support systems that must surface uncertainty before confidence.
- A **creative writing** brain that governs narrative structure, voice, and revision strategy.

Each Pluggable Brain is a JSON file containing rules with the standard schema (name, directive, domain, phase, applies_when, anti_applies_when, strength). Each is loaded via the same retrieval infrastructure — the Closed-Vocabulary Bridge — that loads Knowledge Packs.

If you have expertise in a domain — law, medicine, negotiation, engineering, operations, safety — your Pluggable Brain is the most valuable contribution you can make.

Review the grammar. The 40 primitives — 15 thematic roles, 7 binary relations, 8 link types, 10 frame and group structures — are stable. Do you see a gap? Open an issue. The grammar has survived years of adversarial testing. But it may not cover your domain. If it does not, tell us.

Build an adapter. Write a Knowledge Graph loader for a backend not yet covered. The architecture is backend-agnostic. If your graph database, vector store, or relational database is not yet supported, an adapter is a week of work.

Write a Knowledge Pack. If you are a domain expert in a field with well-defined specifications — medical protocols, regulatory compliance, engineering standards, military doctrine — compile your knowledge into a pack and publish it. The pack format is specified. The tooling exists. Your pack will be used by every organization that loads it.

Deploy a pilot. Pick one system, one integration, one domain. Deploy the Core Lexicon as a shared dictionary hub. Measure the reduction in integration time, the elimination of guessing, the number of GapReports produced instead of silent fabrications. Publish your results.

Join the community. The repository has a discussion forum, a community call schedule, and a contributor guide. The architecture is open. The standards are open. The community is open.

6.9 What Success Looks Like

For an organization deploying autonomous systems: A coalition exercise where a ground station and an allied drone coordinate on first contact. The integration takes seconds, not months. The drone refuses a HIGH_RISK_COMMAND that lacks a human authorization — not because the message was invalid, but because the constitution required it. A tactical-decision Pluggable Brain ensures every drone can refuse unlawful orders without requiring a rules update from command. The refusal produces a GapReport, not a catastrophic failure. If the Wisdom Harvesting pipeline is active, the system learns from every exercise — so the next coalition forms faster.

For a mission architect: A deep space probe that operates autonomously through hours of communication delay. It refuses a power-intensive command when its power budget is already committed. It survives a failure that killed its predecessor, because the predecessor's Lifeboat Pack was loaded before launch. The probe returns data, not silence.

For a supply chain executive: A supplier sends a component specification. The system aligns it deterministically — not with a similarity score, but with a ProofTrace or GapReport. A counterfeit part is detected because its provenance chain has a broken link. Returns are processed in minutes, not weeks, because the system can identify parts without labels. The next supplier integration is faster than the first.

For an industrial robotics operator: A robot refuses a command that would create an unsafe condition. A near-miss is harvested and generates a new safety rule before the next shift. Retooling for a new product line takes hours, not days, because the Knowledge Pack for the new product's safety rules is loaded in seconds.

For everyone: A machine that says "I don't know" instead of fabricating. A machine that can refuse a command that violates its constitution. A machine that inherits the wisdom of its predecessors. A machine whose pluggable brain — its behavioral governance — can be swapped, updated, and improved without touching the underlying model. A machine that compounds its expertise over time. A machine that can be trusted.

6.10 For the Person Who Must Sell This Internally

If you are reading this paper because you need to convince your organization to adopt SGF, here is the language you need.

The one-sentence pitch:

> SGF is an open architecture that lets machines refuse unsafe commands, verify specifications without guessing, and compound their expertise over time — without changing their kernel or violating their constitution.

The ROI framing for a decision-maker:

> The cost of deploying the Core Lexicon is approximately the cost of a single failed integration. The benefit is that every future integration starts from a shared foundation instead of from zero. The break-even point is the second integration.

The risk mitigation framing:

> SGF is published as open standards under a perpetual patent pledge. There is no vendor lock-in. The architecture is the standard, not the implementation. You can change implementations without losing your investment in the knowledge base.

The incremental adoption framing:

> You do not need to deploy the full stack. You can start with the Core Lexicon as a shared dictionary hub. You can add governance later. You can add Wisdom Harvesting when your corpus reaches scale. Every phase delivers independent value.

Closing

Consider the machine that received a command it could not obey. Its constitution required authorization it did not have. It refused. It produced a GapReport. The operator corrected the gap. The mission proceeded.

Every machine that needs to refuse an unsafe command deserves the same architecture. Every integration that currently takes months deserves to take seconds. Every lesson learned in blood deserves to be inherited by the next system that encounters the same pattern. Every system that needs to think clearly, honestly, and creatively deserves a Pluggable Brain that governs its behavior — not just prompts that can be overridden.

The architecture exists. The code is open. The standards are published. The question is not whether it is possible. The question is whether you will help deploy it.

- End of white paper.*